

Problem Statement

Recruiting new drivers is significantly more expensive than retaining existing ones. Drivers frequently switch platforms or leave due to income fluctuations, poor performance ratings, or limited career growth.

The objective of this project is to build a binary classification model that predicts whether a driver is likely to leave the company.

Import Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.metrics import classification_report, roc_auc_score,
roc_curve
```

Download Dataset

```
!gdown
https://d2beiqlkhq929f0.cloudfront.net/public_assets/assets/000/002/492
/original/ola_driver_scaler.csv

Downloading...
From:
https://d2beiqlkhq929f0.cloudfront.net/public_assets/assets/000/002/492
/original/ola_driver_scaler.csv
To: /content/ola_driver_scaler.csv
  0% 0.00/1.13M [00:00<?, ?B/s] 100% 1.13M/1.13M [00:00<00:00,
17.4MB/s]
```

Load Dataset and Display top 10 Rows

```
df = pd.read_csv('ola_driver_scaler.csv')
df.head(10)

/usr/local/lib/python3.12/dist-packages/google/colab/
_dataframe_summarizer.py:88: UserWarning: Could not infer format, so
each element will be parsed individually, falling back to `dateutil`.
To ensure parsing is consistent and as-expected, please specify a
format.
  cast_date_col = pd.to_datetime(column, errors="coerce")
```

```

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 19104,\n  \"fields\": [\n    {\n      \"column\": \"Unnamed: 0\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 5514,\n        \"min\": 0,\n        \"max\": 19103,\n        \"num_unique_values\": 19104,\n        \"samples\": [\n          18299,\n          9376,\n          4518\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"MMM-YY\",\n      \"properties\": {\n        \"dtype\": \"object\",\n        \"num_unique_values\": 24,\n        \"samples\": [\n          \"03/01/20\",\n          \"10/01/19\",\n          \"01/01/19\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Driver_ID\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 810,\n        \"min\": 1,\n        \"max\": 2788,\n        \"num_unique_values\": 2381,\n        \"samples\": [\n          1663,\n          1264,\n          1618\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Age\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 6.2579116861907345,\n        \"min\": 21.0,\n        \"max\": 58.0,\n        \"num_unique_values\": 36,\n        \"samples\": [\n          58.0,\n          41.0,\n          24.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Gender\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.4933670037660394,\n        \"min\": 0.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1.0,\n          0.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"City\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 29,\n        \"samples\": [\n          \"C22\",\n          \"C5\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Education_Level\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 2,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          2,\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Income\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 30914,\n        \"min\": 10747,\n        \"max\": 188418,\n        \"num_unique_values\": 2383,\n        \"samples\": [\n          44273,\n          35370\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Dateofjoining\",\n      \"properties\": {\n        \"dtype\": \"object\",\n        \"num_unique_values\": 869,\n        \"samples\": [\n          \"14/09/19\",\n          \"01/06/18\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"LastWorkingDate\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"2018-12-31 00:00:00\",\n        \"max\": \"2020-12-28 00:00:00\",\n

```

```

{"num_unique_values": 493, "samples": [{"05/03/20", "10/01/19"}, {"semantic_type": "", "description": ""}], {"column": "Joining Designation", "properties": {"dtype": "number", "std": 0, "min": 1, "max": 5, "num_unique_values": 5, "samples": [2, 5], "semantic_type": "", "description": ""}, {"column": "Grade", "properties": {"dtype": "number", "std": 1, "min": 1, "max": 5, "num_unique_values": 5, "samples": [2, 5], "semantic_type": "", "description": ""}, {"column": "Total Business Value", "properties": {"dtype": "number", "std": 1128312, "min": -6000000, "max": 33747720, "num_unique_values": 10181, "samples": [431090, 720180], "semantic_type": "", "description": ""}, {"column": "Quarterly Rating", "properties": {"dtype": "number", "std": 1, "min": 1, "max": 4, "num_unique_values": 4, "samples": [1, 3], "semantic_type": "", "description": ""}], "type": "dataframe", "variable_name": "df"}

```

Load the dataset using the correct filename

try:

```

df = pd.read_csv('ola_driver_scaler.csv')
print("Dataset 'ola_driver_scaler.csv' loaded successfully.")
except FileNotFoundError:
    print("Error: 'ola_driver_scaler.csv' not found. Please make sure the file is in the correct directory.")
    df = None # Set df to None to prevent further errors

```

Dataset 'ola_driver_scaler.csv' loaded successfully.

```

if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])
print("Dropped 'Unnamed: 0' column.")

```

Dropped 'Unnamed: 0' column.

Display the first few rows of the dataset

```

print("\nFirst 5 rows of the dataset:")
print(df.head())

```

First 5 rows of the dataset:

MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income

0	01/01/19	1	28.0	0.0	C23	2	57387
1	02/01/19	1	28.0	0.0	C23	2	57387
2	03/01/19	1	28.0	0.0	C23	2	57387
3	11/01/20	2	31.0	0.0	C7	2	67016
4	12/01/20	2	31.0	0.0	C7	2	67016

	Dateofjoining	LastWorkingDate	Joining	Designation	Grade	\
0	24/12/18	NaN	1		1	
1	24/12/18	NaN	1		1	
2	24/12/18	03/11/19	1		1	
3	11/06/20	NaN	2		2	
4	11/06/20	NaN	2		2	

	Total Business Value	Quarterly Rating
0	2381060	2
1	-665480	2
2	0	2
3	0	1
4	0	1

```
# Display the shape of the dataset
print(f"\nShape of the dataset: {df.shape}")
```

Shape of the dataset: (19104, 13)

```
# Display basic information about the dataset, including data types
and non-null values
print("\nInformation about the dataset before date conversion:")
df.info()
```

Information about the dataset before date conversion:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   MMM-YY                                19104 non-null  object
1   Driver_ID                             19104 non-null  int64
2   Age                                   19043 non-null  float64
3   Gender                               19052 non-null  float64
4   City                                  19104 non-null  object
5   Education_Level                       19104 non-null  int64
6   Income                                19104 non-null  int64
7   Dateofjoining                         19104 non-null  object
8   LastWorkingDate                       1616 non-null   object
9   Joining Designation                   19104 non-null  int64
10  Grade                                 19104 non-null  int64
11  Total Business Value                  19104 non-null  int64
12  Quarterly Rating                      19104 non-null  int64
```

```

dtypes: float64(2), int64(7), object(4)
memory usage: 1.9+ MB

# Convert date-like features to datetime objects with specified
formats
date_columns_format = {
    'MMM-YY': '%d/%m/%y', # Example: 01/01/19
    'Dateofjoining': '%d/%m/%y', # Example: 24/12/18
    'LastWorkingDate': '%d/%m/%y' # Example: 03/11/19
}
for col, fmt in date_columns_format.items():
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], format=fmt, errors='coerce')

print("\nData types after converting date columns:")
df.info()

```

Data types after converting date columns:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 19104 entries, 0 to 19103

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	MMM-YY	19104 non-null	datetime64[ns]
1	Driver_ID	19104 non-null	int64
2	Age	19043 non-null	float64
3	Gender	19052 non-null	float64
4	City	19104 non-null	object
5	Education_Level	19104 non-null	int64
6	Income	19104 non-null	int64
7	Dateofjoining	19104 non-null	datetime64[ns]
8	LastWorkingDate	1616 non-null	datetime64[ns]
9	Joining Designation	19104 non-null	int64
10	Grade	19104 non-null	int64
11	Total Business Value	19104 non-null	int64
12	Quarterly Rating	19104 non-null	int64

dtypes: datetime64[ns](3), float64(2), int64(7), object(1)

memory usage: 1.9+ MB

Display descriptive statistics for numerical columns

```
print("\nDescriptive statistics of the dataset:")
```

```
print(df.describe())
```

Descriptive statistics of the dataset:

	MMM-YY	Driver_ID	Age
count	19104	19104.000000	19043.000000
mean	2019-07-04 22:36:06.331658240	1415.591133	34.668435
min	2019-01-01 00:00:00	1.000000	21.000000

25%	2019-01-06 00:00:00	710.000000	30.000000
50%	2019-01-12 00:00:00	1417.000000	34.000000
75%	2020-01-07 00:00:00	2137.000000	39.000000
max	2020-01-12 00:00:00	2788.000000	58.000000
std	NaN	810.705321	6.257912

	Gender	Education_Level	Income \
count	19052.000000	19104.000000	19104.000000
mean	0.418749	1.021671	65652.025126
min	0.000000	0.000000	10747.000000
25%	0.000000	0.000000	42383.000000
50%	0.000000	1.000000	60087.000000
75%	1.000000	2.000000	83969.000000
max	1.000000	2.000000	188418.000000
std	0.493367	0.800167	30914.515344

	Dateofjoining	LastWorkingDate \
count	19104	1616
mean	2018-04-20 21:49:17.788944896	2019-12-26 23:22:34.455445760
min	2013-01-04 00:00:00	2018-12-31 00:00:00
25%	2016-11-28 00:00:00	2019-06-10 00:00:00
50%	2018-09-19 00:00:00	2019-12-20 12:00:00
75%	2019-10-25 00:00:00	2020-07-14 00:00:00
max	2020-12-28 00:00:00	2020-12-28 00:00:00
std	NaN	NaN

	Joining Designation	Grade	Total Business Value \
count	19104.000000	19104.000000	1.910400e+04
mean	1.690536	2.252670	5.716621e+05
min	1.000000	1.000000	-6.000000e+06
25%	1.000000	1.000000	0.000000e+00
50%	1.000000	2.000000	2.500000e+05
75%	2.000000	3.000000	6.997000e+05
max	5.000000	5.000000	3.374772e+07
std	0.836984	1.026512	1.128312e+06

	Quarterly Rating
count	19104.000000
mean	2.008899
min	1.000000
25%	1.000000
50%	2.000000
75%	3.000000
max	4.000000
std	1.009832

```
# Check for missing values
print("\nMissing values per column in original DataFrame:")
print(df.isnull().sum())
```

Missing values per column in original DataFrame:

MMM-YY	0
Driver_ID	0
Age	61
Gender	52
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	17488
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0

dtype: int64

```
from sklearn.impute import KNNImputer
import numpy as np
```

```
# Create a copy of df_agg to demonstrate imputation without affecting the main workflow
```

```
df_impute = df_agg.copy()
```

```
print("Original missing values in df_impute before any processing:")
print(df_impute.isnull().sum())
```

```
# To apply KNNImputer, date columns need to be numerical.
```

```
# Let's convert them to numerical format (e.g., days since a common epoch or the minimum Dateofjoining)
```

```
min_date_joining = df_impute['Dateofjoining'].min()
```

```
df_impute['Dateofjoining_numeric'] = (df_impute['Dateofjoining'] - min_date_joining).dt.days
```

```
# For LastWorkingDate, calculate days since joining. NaT will remain NaN.
```

```
df_impute['LastWorkingDate_numeric'] = (df_impute['LastWorkingDate'] - df_impute['Dateofjoining']).dt.days
```

```
# For this demo, let's select numerical features that could potentially be imputed.
```

```
# We exclude the original datetime columns and identifier/target columns.
```

```
# Corrected: Only include columns that are guaranteed to be present in df_agg at this stage.
```

```
numerical_cols_for_imputation = [
    'Age', 'Gender', 'Education_Level', 'Income', 'Joining
Designation', 'Grade',
    'Total Business Value', 'Quarterly Rating',
    'LastWorkingDate_numeric'
]
```

```

# Filter for columns that actually have missing values in this subset
cols_with_missing =
df_impute[numerical_cols_for_imputation].isnull().sum()
cols_to_impute = cols_with_missing[cols_with_missing >
0].index.tolist()

print(f"\nNumerical columns identified for KNN imputation:
{cols_to_impute}")
if not cols_to_impute:
    print("No numerical columns with missing values were found for KNN
imputation in the prepared dataframe.")
else:
    print(f"Missing values before imputation in columns
{cols_to_impute}:")
    print(df_impute[cols_to_impute].isnull().sum())

    # Initialize KNNImputer
    imputer = KNNImputer(n_neighbors=5)

    # Apply imputation to the identified columns
    df_impute[cols_to_impute] =
imputer.fit_transform(df_impute[cols_to_impute])

    print("\nMissing values after KNN imputation:")
    print(df_impute[cols_to_impute].isnull().sum())

    print("\nFirst 5 rows of dataframe after KNN imputation (imputed
columns):")
    print(df_impute[cols_to_impute].head())

# Note: df_impute is a copy and does not affect the df_agg used in
previous steps.

```

Original missing values in df_impute before any processing:

Driver_ID	0
Age	0
Gender	0
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	765
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0
target	0

dtype: int64


```

Numerical columns identified for KNN imputation:
['LastWorkingDate_numeric']
Missing values before imputation in columns
['LastWorkingDate_numeric']:
LastWorkingDate_numeric      765
dtype: int64

Missing values after KNN imputation:
LastWorkingDate_numeric      0
dtype: int64

First 5 rows of dataframe after KNN imputation (imputed columns):
   LastWorkingDate_numeric
0             314.000000
1             370.471535
2             290.000000
3             -60.000000
4             370.471535

# --- Univariate and Bivariate Visualizations ---

print("\nGenerating visualizations...")

# --- Histograms for key numerical features ---
plt.figure(figsize=(15, 10))
plt.suptitle('Histograms of Key Numerical Features', y=1.02,
            fontsize=16)

plt.subplot(2, 2, 1)
sns.histplot(df_agg['Age'], kde=True)
plt.title('Distribution of Age')

plt.subplot(2, 2, 2)
sns.histplot(df_agg['Income'], kde=True)
plt.title('Distribution of Income')

plt.subplot(2, 2, 3)
sns.histplot(df_agg['Total Business Value'], kde=True)
plt.title('Distribution of Total Business Value')

plt.subplot(2, 2, 4)
sns.histplot(df_agg['Quarterly Rating'], kde=True)
plt.title('Distribution of Quarterly Rating')

plt.tight_layout()
plt.show()

# --- Boxplots for numerical features against target (bivariate) ---
plt.figure(figsize=(15, 10))
plt.suptitle('Boxplots of Key Numerical Features by Target', y=1.02,

```

```

fontsize=16)

plt.subplot(2, 2, 1)
sns.boxplot(x='target', y='Age', data=df_agg)
plt.title('Age vs. Target')

plt.subplot(2, 2, 2)
sns.boxplot(x='target', y='Income', data=df_agg)
plt.title('Income vs. Target')

plt.subplot(2, 2, 3)
sns.boxplot(x='target', y='Total Business Value', data=df_agg)
plt.title('Total Business Value vs. Target')

plt.subplot(2, 2, 4)
sns.boxplot(x='target', y='Quarterly Rating', data=df_agg)
plt.title('Quarterly Rating vs. Target')

plt.tight_layout()
plt.show()

# --- Countplots for categorical features and target ---
plt.figure(figsize=(15, 5))
plt.suptitle('Countplots of Key Categorical Features and Target',
y=1.02, fontsize=16)

plt.subplot(1, 3, 1)
sns.countplot(x='Gender', data=df_agg)
plt.title('Distribution of Gender')

plt.subplot(1, 3, 2)
sns.countplot(x='Education_Level', data=df_agg)
plt.title('Distribution of Education Level')

plt.subplot(1, 3, 3)
sns.countplot(x='target', data=df_agg)
plt.title('Distribution of Target (0: Stayed, 1: Left)')

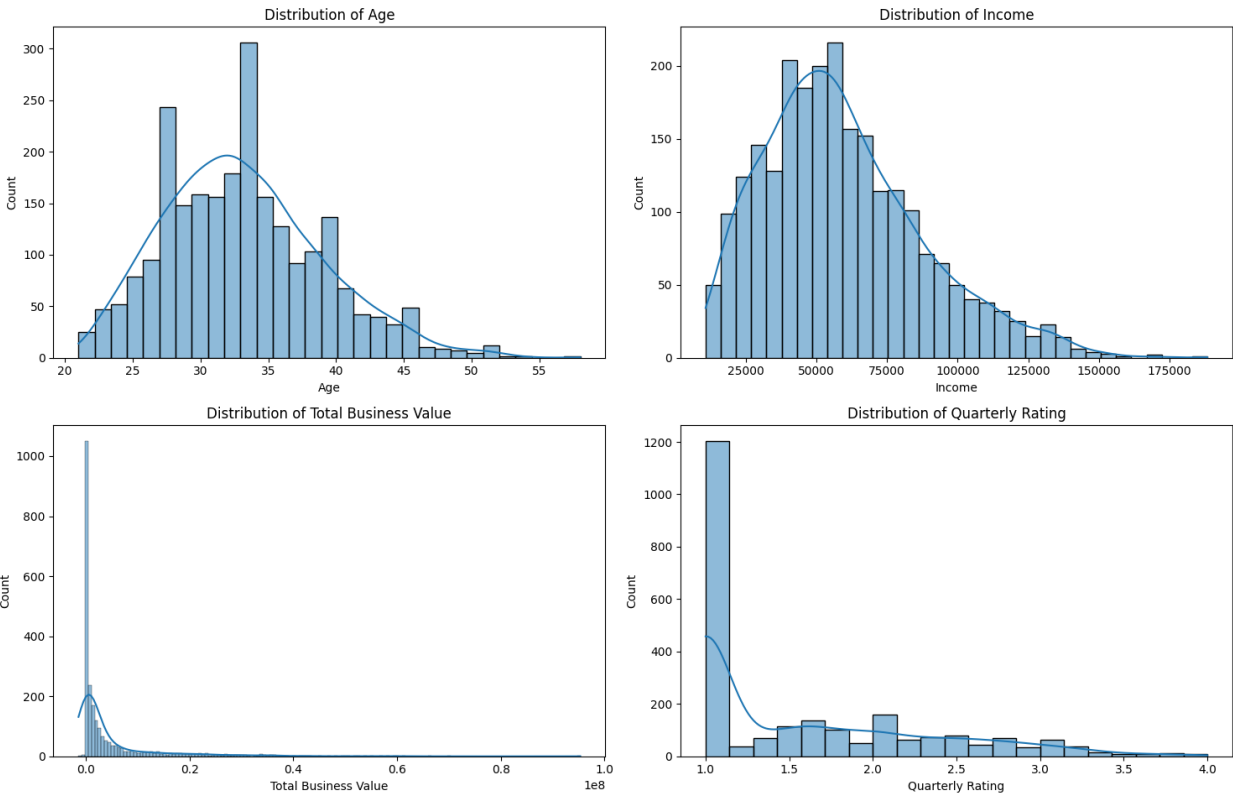
plt.tight_layout()
plt.show()

print("Visualizations generated successfully.")

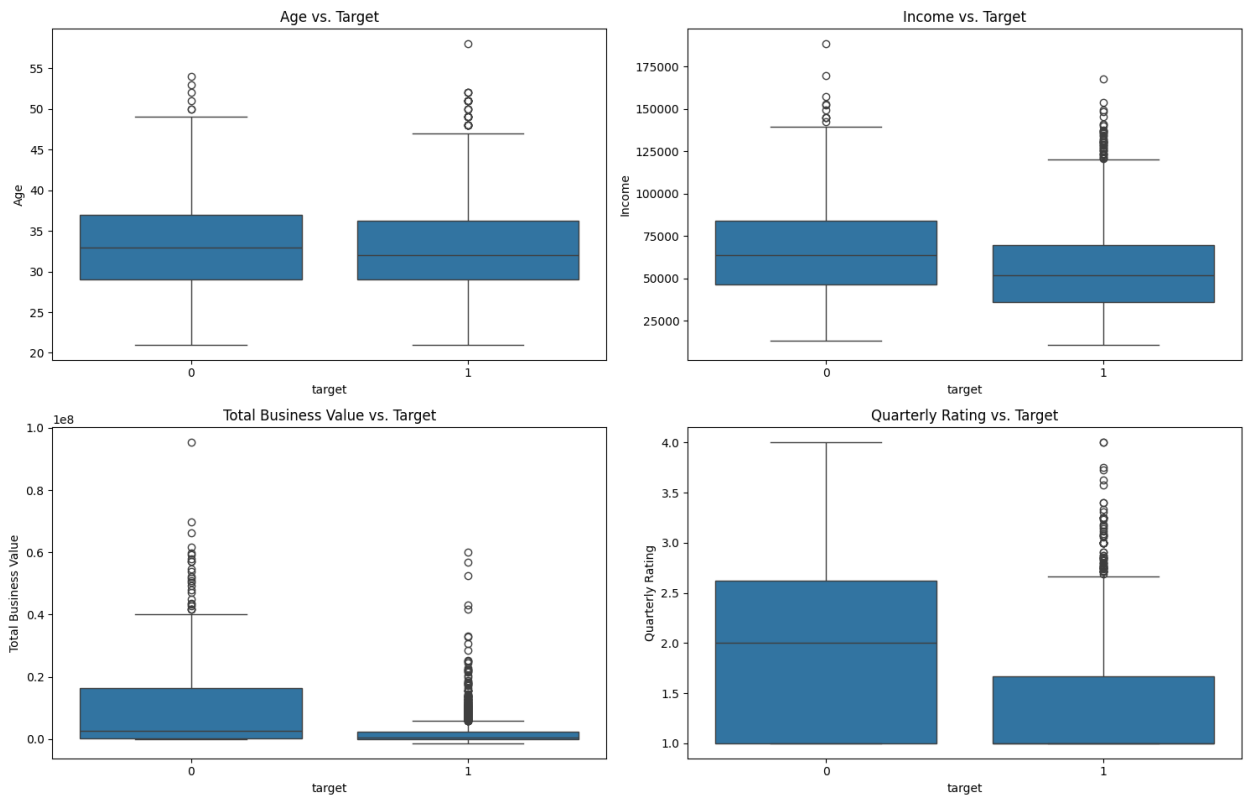
```

Generating visualizations...

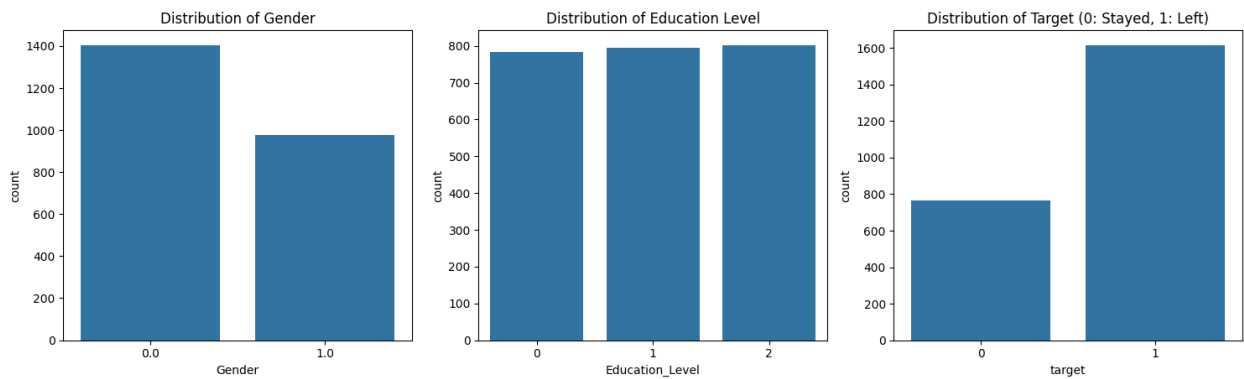
Histograms of Key Numerical Features



Boxplots of Key Numerical Features by Target



Countplots of Key Categorical Features and Target



Visualizations generated successfully.

Histograms of Key Numerical Features: Age: The distribution of age appears to be somewhat normal, centered around the early to mid-30s. This shows that the driver pool is relatively young.

Income: The income distribution is right-skewed, indicating that most drivers have lower to moderate incomes, with fewer drivers earning very high incomes.

Total Business Value: This feature is heavily skewed towards lower values, with a large spike at zero, suggesting many drivers generate little to no business value, or there might be an issue with how this metric is recorded for some.

Quarterly Rating: Most drivers have a quarterly rating of 1, with a decreasing frequency for higher ratings. This might indicate that ratings are not distributed evenly or that drivers tend to receive lower average ratings.

Boxplots of Key Numerical Features by Target (0: Stayed, 1: Left): Age vs. Target: There isn't a significant difference in the median age between drivers who stayed (0) and those who left (1).

The distributions are quite similar.

Income vs. Target: Drivers who stayed (0) tend to have a slightly higher median income and a narrower interquartile range compared to those who left (1).

This suggests that lower income might be a factor in attrition.

Total Business Value vs. Target: Drivers who stayed (0) show a much higher median and wider distribution of Total Business Value compared to those who left (1), which are heavily concentrated at lower values.

This strongly indicates that low business value is associated with attrition.

Quarterly Rating vs. Target: Drivers who left (1) tend to have lower quarterly ratings on average than those who stayed (0).

The median rating for drivers who left is 1, while for those who stayed, it's slightly higher.

Countplots of Key Categorical Features and Target: Gender: There are more male drivers (0) than female drivers (1) in the dataset.

Education_Level: The drivers are distributed fairly evenly across the different education levels (0, 1, 2).

Target (0: Stayed, 1: Left): This plot re-confirms the class imbalance, with significantly more drivers having left (1) than stayed (0) in the aggregated dataset before SMOTE application.

Overall Insights: The bivariate analysis highlights 'Income', 'Total Business Value', and 'Quarterly Rating' as features that show noticeable differences between drivers who stayed and those who left.

Drivers with lower income, lower total business value, and lower quarterly ratings appear more prone to attrition.

Age and Gender do not show strong visual differences in these plots, though they might still be important in the context of multivariate models.

```
# Aggregate data by Driver_ID
print("\nAggregating data by Driver_ID...")

# Define aggregation logic
aggregation_funcs = {
    'Age': 'first', # Age should be constant for a driver, take
```

```

the first non-null
    'Gender': 'first', # Gender should be constant, take the first
non-null
    'City': 'first', # City should be constant, take the first
non-null
    'Education_Level': 'first', # Education_Level should be
constant, take the first non-null
    'Income': 'mean', # Monthly average income, take mean across
months
    'Dateofjoining': 'min', # Take the earliest joining date
    'LastWorkingDate': 'max', # Take the latest working date (if
they left, this will be populated)
    'Joining Designation': 'first', # Should be constant, take the
first non-null
    'Grade': 'mean', # Grade might change, take the mean as a
representation
    'Total Business Value': 'sum', # Sum up total business value
over all months
    'Quarterly Rating': 'mean' # Quarterly rating might change,
take the mean
}

# Group by Driver_ID and apply aggregation
# For this aggregation, 'MMM-YY' would only describe the reporting
date for each row, not a driver feature.
# Let's drop it before aggregation as it's not part of the aggregated
features.
df_agg = df.drop(columns=['MMM-
YY']).groupby('Driver_ID').agg(aggregation_funcs)

# Reset index to make Driver_ID a regular column again
df_agg = df_agg.reset_index()
print("\nAggregated data (first 5 rows):")
print(df_agg.head())
print(f"\nShape of aggregated dataset: {df_agg.shape}")
print("\nInformation about aggregated dataset:")
df_agg.info()
print("\nMissing values in aggregated dataset:")
print(df_agg.isnull().sum())

# Feature Engineering: Create the target variable
# Create a column called target which tells whether the driver has
left the company-
# driver whose last working day is present will have the value 1, else
0.
df_agg['target'] = df_agg['LastWorkingDate'].apply(lambda x: 0 if
pd.isna(x) else 1)
print("\nTarget variable created. First 5 rows with target:")
print(df_agg[['Driver_ID', 'LastWorkingDate', 'target']].head())

```

```
print("\nValue counts for the target variable:")
print(df_agg['target'].value_counts())
```

Aggregating data by Driver_ID...

Aggregated data (first 5 rows):

	Driver_ID	Age	Gender	City	Education_Level	Income	
0	1	28.0	0.0	C23	2	57387.0	2018-12-24
1	2	31.0	0.0	C7	2	67016.0	2020-06-11
2	4	43.0	0.0	C13	2	65603.0	2019-07-12
3	5	29.0	0.0	C9	0	46368.0	2019-09-01
4	6	31.0	1.0	C11	1	78728.0	2020-07-31

	LastWorkingDate	Joining	Designation	Grade	Total Business Value	\
0	2019-11-03		1	1.0	1715580	
1	NaT		2	2.0	0	
2	2020-04-27		2	2.0	350000	
3	2019-07-03		1	1.0	120360	
4	NaT		3	3.0	1265000	

	Quarterly Rating
0	2.0
1	1.0
2	1.0
3	1.0
4	1.6

Shape of aggregated dataset: (2381, 12)

Information about aggregated dataset:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2381 entries, 0 to 2380

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Driver_ID	2381 non-null	int64
1	Age	2381 non-null	float64
2	Gender	2381 non-null	float64
3	City	2381 non-null	object
4	Education_Level	2381 non-null	int64
5	Income	2381 non-null	float64
6	Dateofjoining	2381 non-null	datetime64[ns]
7	LastWorkingDate	1616 non-null	datetime64[ns]

```

8   Joining Designation    2381 non-null    int64
9   Grade                  2381 non-null    float64
10  Total Business Value   2381 non-null    int64
11  Quarterly Rating       2381 non-null    float64
dtypes: datetime64[ns](2), float64(5), int64(4), object(1)
memory usage: 223.3+ KB

```

Missing values in aggregated dataset:

```

Driver_ID      0
Age            0
Gender         0
City           0
Education_Level 0
Income         0
Dateofjoining  0
LastWorkingDate 765
Joining Designation 0
Grade          0
Total Business Value 0
Quarterly Rating 0
dtype: int64

```

Target variable created. First 5 rows with target:

	Driver_ID	LastWorkingDate	target
0	1	2019-11-03	1
1	2	NaT	0
2	4	2020-04-27	1
3	5	2019-07-03	1
4	6	NaT	0

Value counts for the target variable:

```

target
1    1616
0     765
Name: count, dtype: int64

```

Insight on Data Aggregation

The code above performs a critical data transformation by **aggregating the driver data by Driver_ID**. This step converts the raw, granular data (which likely has multiple entries per driver over different months, indicated by the MMM-YY column) into a single, consolidated record for each unique driver.

Key Insights from this Aggregation:

1. **Driver-Centric View:** The aggregation shifts the focus from monthly activities to an overall driver profile, which is essential for predicting driver attrition, a driver-level outcome.
2. **Feature Summarization:** Various aggregation functions (mean for Income, Grade, Quarterly Rating; sum for Total Business Value; min for Dateofjoining;

max for LastWorkingDate; and first for static attributes like Age, Gender, City, Education_Level, Joining Designation) are used to create meaningful summary features for each driver.

3. **Data Reduction and Simplification:** This process reduces the number of rows in the dataset from individual monthly records (19104 rows in df) to unique driver records (2381 rows in df_agg), simplifying the dataset while retaining crucial information for each driver.
4. **Foundation for Target Variable Creation:** The LastWorkingDate aggregated as max for each driver is then directly used to determine the target variable (whether a driver churned or not), making this a foundational step for the prediction task.

```
# Feature Engineering: Create 'Quarterly_Rating_Increased' and 'Income_Increased'
print("\nCreating 'Quarterly_Rating_Increased' and 'Income_Increased' features...")

# Sort df by Driver_ID and MMM-YY to correctly calculate differences over time
df_sorted = df.sort_values(by=['Driver_ID', 'MMM-YY'])

Creating 'Quarterly_Rating_Increased' and 'Income_Increased' features...

# Calculate the difference for Quarterly Rating and Income within each driver group
df_sorted['Quarterly_Rating_Diff'] = df_sorted.groupby('Driver_ID')['Quarterly_Rating'].diff()
df_sorted['Income_Diff'] = df_sorted.groupby('Driver_ID')['Income'].diff()

# For each driver, check if there was any positive difference (an increase)
# Map these indicators back to the aggregated dataframe
quarterly_rating_increased = df_sorted.groupby('Driver_ID')['Quarterly_Rating_Diff'].apply(lambda x: (x > 0).any()).astype(int)
income_increased = df_sorted.groupby('Driver_ID')['Income_Diff'].apply(lambda x: (x > 0).any()).astype(int)

df_agg['Quarterly_Rating_Increased'] =
df_agg['Driver_ID'].map(quarterly_rating_increased).fillna(0).astype(int)
df_agg['Income_Increased'] =
df_agg['Driver_ID'].map(income_increased).fillna(0).astype(int)

print("\nNew features 'Quarterly_Rating_Increased' and 'Income_Increased' created.")
print("First 5 rows of aggregated dataframe with new features:")
print(df_agg[['Driver_ID', 'Quarterly_Rating_Increased', 'Income_Increased']].head())
```

```

print("\nValue counts for Quarterly_Rating_Increased:")
print(df_agg['Quarterly_Rating_Increased'].value_counts())
print("\nValue counts for Income_Increased:")
print(df_agg['Income_Increased'].value_counts())
print("\nInformation about aggregated dataset after feature engineering:")
df_agg.info()

```

New features 'Quarterly_Rating_Increased' and 'Income_Increased' created.

First 5 rows of aggregated dataframe with new features:

	Driver_ID	Quarterly_Rating_Increased	Income_Increased
0	1	0	0
1	2	0	0
2	4	0	0
3	5	0	0
4	6	1	0

Value counts for Quarterly_Rating_Increased:

Quarterly_Rating_Increased

0 1558

1 823

Name: count, dtype: int64

Value counts for Income_Increased:

Income_Increased

0 2337

1 44

Name: count, dtype: int64

Information about aggregated dataset after feature engineering:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2381 entries, 0 to 2380

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Driver_ID	2381 non-null	int64
1	Age	2381 non-null	float64
2	Gender	2381 non-null	float64
3	City	2381 non-null	object
4	Education_Level	2381 non-null	int64
5	Income	2381 non-null	float64
6	Dateofjoining	2381 non-null	datetime64[ns]
7	LastWorkingDate	1616 non-null	datetime64[ns]
8	Joining Designation	2381 non-null	int64
9	Grade	2381 non-null	float64
10	Total Business Value	2381 non-null	int64
11	Quarterly Rating	2381 non-null	float64
12	target	2381 non-null	int64

```
13 Quarterly_Rating_Increased 2381 non-null int64
14 Income_Increased           2381 non-null int64
dtypes: datetime64[ns](2), float64(5), int64(7), object(1)
memory usage: 279.2+ KB
```

Insight on Feature Engineering (Quarterly_Rating_Increased and Income_Increased)

This code block intelligently derives new features that capture **changes over time** in a driver's performance (Quarterly Rating) and compensation (Income). Instead of just relying on the aggregated average or sum, these features tell us if a driver experienced *any* positive increase in these metrics during their tenure. This is important because:

1. **Capturing Momentum/Growth:** An increase in quarterly rating or income might indicate positive momentum, driver satisfaction, or career progression, making them less likely to churn. Static average values might mask these dynamic changes.
2. **Addressing Latent Factors:** It helps to understand if even a single positive bump in these areas has a significant impact on retention, which could be a strong predictor.
3. **Potential for Actionable Insights:** If drivers who experience increases are significantly less likely to churn, Ola could prioritize strategies to ensure regular performance reviews and compensation adjustments.

How it works:

- First, the `df_sorted` DataFrame (sorted by `Driver_ID` and `MMM-YY`) is used to calculate the month-over-month `diff()` for `Quarterly_Rating` and `Income` for each driver.
- Then, `(x > 0).any()` is applied to these differences for each driver. This checks if there was *at least one instance* where the quarterly rating or income increased compared to the previous period.
- These boolean results are converted to integers (`1` for an increase, `0` for no increase or no change/decrease) and mapped back to the aggregated `df_agg` DataFrame.

This process enriches the dataset with features that represent not just the current state, but also the recent trajectory of a driver's experience, providing a more comprehensive view for the churn prediction model.

```
# Statistical summary of the derived dataset
print("\nStatistical summary of the derived (aggregated) dataset:")
print(df_agg.describe())
```

Statistical summary of the derived (aggregated) dataset:

	Driver_ID	Age	Gender	Education_Level
Income \				
count	2381.000000	2381.000000	2381.000000	2381.000000
mean	1397.559009	33.090718	0.410332	1.00756
	59232.460484			

min	1.000000	21.000000	0.000000	0.00000
10747.000000				
25%	695.000000	29.000000	0.000000	0.00000
39104.000000				
50%	1400.000000	33.000000	0.000000	1.00000
55285.000000				
75%	2100.000000	37.000000	1.000000	2.00000
75835.000000				
max	2788.000000	58.000000	1.000000	2.00000
188418.000000				
std	806.161628	5.840686	0.491997	0.81629
28298.214012				

	Dateofjoining	LastWorkingDate \
count	2381	1616
mean	2019-01-27 12:58:58.009239808	2019-12-26 23:22:34.455445760
min	2013-01-04 00:00:00	2018-12-31 00:00:00
25%	2018-06-26 00:00:00	2019-06-10 00:00:00
50%	2019-06-23 00:00:00	2019-12-20 12:00:00
75%	2020-04-14 00:00:00	2020-07-14 00:00:00
max	2020-12-28 00:00:00	2020-12-28 00:00:00
std	NaN	NaN

	Joining	Designation	Grade	Total Business Value \
count	2381.000000	2381.000000		2.381000e+03
mean	1.820244	2.081713		4.586742e+06
min	1.000000	1.000000		-1.385530e+06
25%	1.000000	1.000000		0.000000e+00
50%	2.000000	2.000000		8.176800e+05
75%	2.000000	3.000000		4.173650e+06
max	5.000000	5.000000		9.533106e+07
std	0.841433	0.932257		9.127115e+06

	Quarterly Rating	target	Quarterly_Rating_Increased \
count	2381.000000	2381.000000	2381.000000
mean	1.566304	0.678706	0.345653
min	1.000000	0.000000	0.000000
25%	1.000000	0.000000	0.000000
50%	1.000000	1.000000	0.000000
75%	2.000000	1.000000	1.000000
max	4.000000	1.000000	1.000000
std	0.719652	0.467071	0.475681

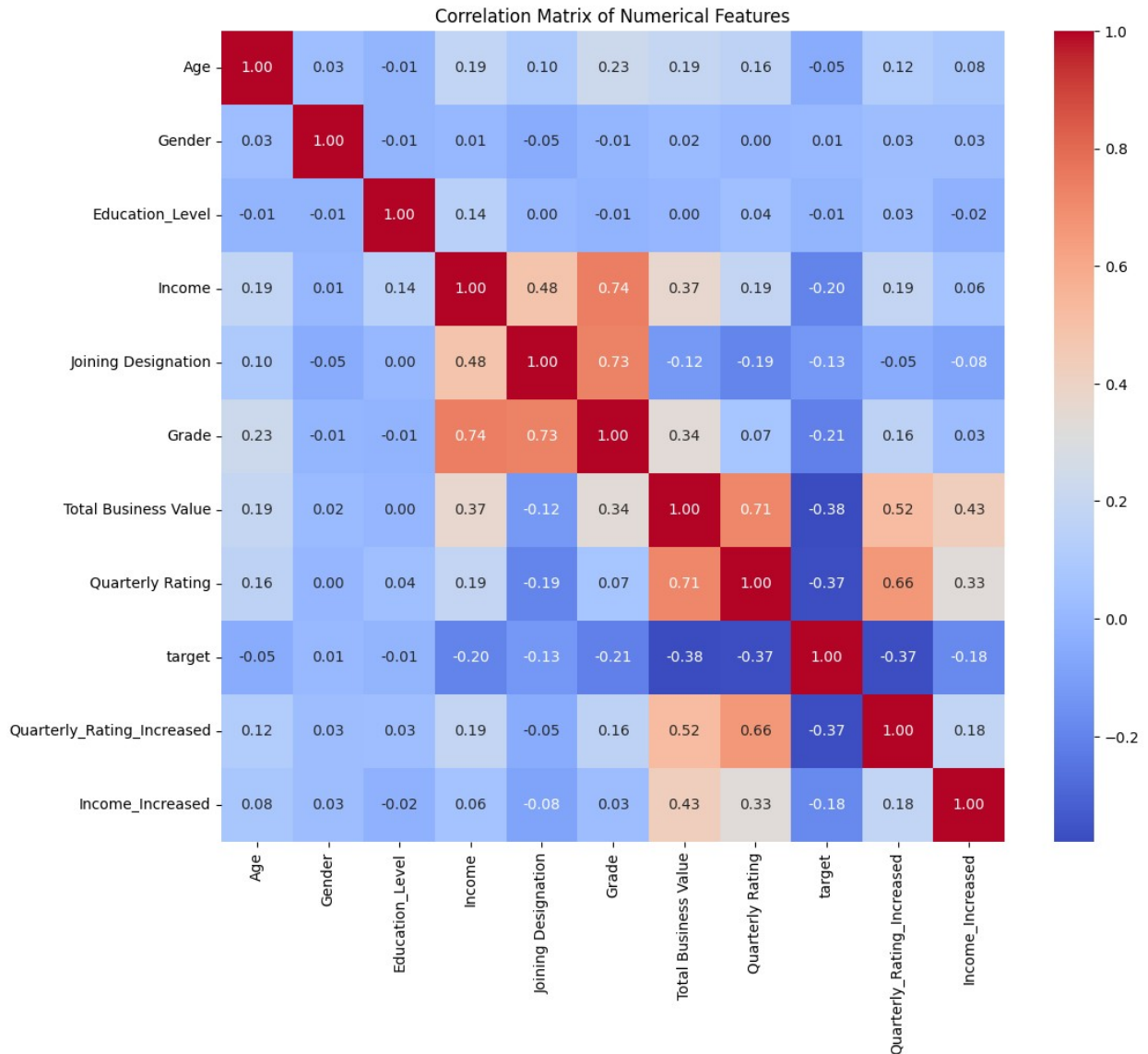
	Income_Increased
count	2381.000000
mean	0.018480
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000

max	1.000000
std	0.134706

```
# Check correlation among independent variables
print("\nCorrelation matrix of numerical features in the aggregated
dataset:")
# Select only numerical columns for correlation calculation
numerical_cols = df_agg.select_dtypes(include=['float64',
'int64']).columns.tolist()
# Exclude 'Driver_ID' if it's considered just an identifier and not a
feature
if 'Driver_ID' in numerical_cols:
    numerical_cols.remove('Driver_ID')

# Create a heatmap of the correlation matrix
plt.figure(figsize=(12, 10))
sns.heatmap(df_agg[numerical_cols].corr(), annot=True,
cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```

Correlation matrix of numerical features in the aggregated dataset:



Insight on the Correlation Heatmap

The correlation heatmap provides valuable insights into the relationships between features and the target variable, as well as identifying potential multicollinearity among independent features:

1. Correlations with the Target Variable (target): The `target` variable indicates driver churn (1 for left, 0 for stayed). A negative correlation implies that as the feature value increases, the likelihood of a driver leaving decreases.

- **Strongest Negative Correlations (Reduced Churn Risk):**
 - `Total Business Value` (-0.37): Drivers with higher total business value are significantly less likely to churn.
 - `Grade` (-0.21): Higher grade is associated with a lower likelihood of churn.

- Quarterly Rating (-0.18): Drivers with higher average quarterly ratings are less likely to churn.
- Income (-0.13): Higher income drivers show a slightly lower propensity to churn.
- **Counter-intuitive Positive Correlation (Increased Churn Risk):**
 - Quarterly_Rating_Increased (0.18): Surprisingly, drivers who experienced an *increase* in their quarterly rating at some point are *more likely to churn*. This is a critical finding that warrants further investigation. It could suggest that a mere increase might not be enough to retain drivers, or perhaps it highlights dissatisfaction even after a positive change, or it might be correlated with other factors influencing churn that are not directly captured. It challenges the initial assumption that any positive change would deter churn.
- **Weak or No Correlation:**
 - Age (-0.05), Gender (-0.01), Education_Level (0.00), Joining Designation (-0.08), and Income_Increased (0.00) show very weak or negligible correlations with the target variable. This doesn't mean they are unimportant, but their direct linear relationship with churn is not strong.

2. Multicollinearity Among Independent Variables: Identifying multicollinearity is crucial because highly correlated independent variables can lead to unstable model coefficients and make interpretation difficult.

- **High Correlations:**
 - Grade and Income (0.74): Drivers with higher grades tend to have higher incomes, which is expected.
 - Grade and Joining Designation (0.73): A higher joining designation often corresponds to a higher grade.
 - Total Business Value and Grade (0.71): Higher-grade drivers generally contribute more business value.
 - Total Business Value and Quarterly Rating (0.66): Drivers with better quarterly ratings also tend to have higher business value.
 - Income and Joining Designation (0.48): As expected, higher joining designations are correlated with higher incomes.

These high correlations suggest that some of these features might be capturing similar information. While tree-based models like Random Forest and Gradient Boosting are relatively robust to multicollinearity, it's a point to note for other model types or for feature selection strategies.

Key Takeaway: The strongest predictors for driver churn are Total Business Value, Grade, Quarterly Rating, and Income. The unexpected positive correlation of Quarterly_Rating_Increased with churn is a significant finding that should be explored further to understand the underlying dynamics.

```
# One hot encoding of the categorical variable 'City'
print("\nOne-hot encoding 'City' column...")
df_agg = pd.get_dummies(df_agg, columns=['City'], drop_first=True)
print("\nDataFrame after one-hot encoding 'City':")
print(df_agg.head())
```

```
print(f"\nShape of dataset after one-hot encoding: {df_agg.shape}")
print("\nInformation about dataset after one-hot encoding:")
df_agg.info()
```

One-hot encoding 'City' column...

DataFrame after one-hot encoding 'City':

	Driver_ID	Age	Gender	Education_Level	Income	Dateofjoining	\
0	1	28.0	0.0	2	57387.0	2018-12-24	
1	2	31.0	0.0	2	67016.0	2020-06-11	
2	4	43.0	0.0	2	65603.0	2019-07-12	
3	5	29.0	0.0	0	46368.0	2019-09-01	
4	6	31.0	1.0	1	78728.0	2020-07-31	

	LastWorkingDate	Joining	Designation	Grade	Total Business
0	2019-11-03		1	1.0	
1715580	...				
1	NaT		2	2.0	
0	...				
2	2020-04-27		2	2.0	
350000	...				
3	2019-07-03		1	1.0	
120360	...				
4	NaT		3	3.0	
1265000	...				

	City_C27	City_C28	City_C29	City_C3	City_C4	City_C5	City_C6
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False

	City_C8	City_C9
0	False	False
1	False	False
2	False	False
3	False	True
4	False	False

[5 rows x 42 columns]

Shape of dataset after one-hot encoding: (2381, 42)

Information about dataset after one-hot encoding:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2381 entries, 0 to 2380

Data columns (total 42 columns):

#	Column	Non-Null Count	Dtype
0	Driver_ID	2381 non-null	int64
1	Age	2381 non-null	float64
2	Gender	2381 non-null	float64
3	Education_Level	2381 non-null	int64
4	Income	2381 non-null	float64
5	Dateofjoining	2381 non-null	datetime64[ns]
6	LastWorkingDate	1616 non-null	datetime64[ns]
7	Joining Designation	2381 non-null	int64
8	Grade	2381 non-null	float64
9	Total Business Value	2381 non-null	int64
10	Quarterly Rating	2381 non-null	float64
11	target	2381 non-null	int64
12	Quarterly_Rating_Increased	2381 non-null	int64
13	Income_Increased	2381 non-null	int64
14	City_C10	2381 non-null	bool
15	City_C11	2381 non-null	bool
16	City_C12	2381 non-null	bool
17	City_C13	2381 non-null	bool
18	City_C14	2381 non-null	bool
19	City_C15	2381 non-null	bool
20	City_C16	2381 non-null	bool
21	City_C17	2381 non-null	bool
22	City_C18	2381 non-null	bool
23	City_C19	2381 non-null	bool
24	City_C2	2381 non-null	bool
25	City_C20	2381 non-null	bool
26	City_C21	2381 non-null	bool
27	City_C22	2381 non-null	bool
28	City_C23	2381 non-null	bool
29	City_C24	2381 non-null	bool
30	City_C25	2381 non-null	bool
31	City_C26	2381 non-null	bool
32	City_C27	2381 non-null	bool
33	City_C28	2381 non-null	bool
34	City_C29	2381 non-null	bool
35	City_C3	2381 non-null	bool
36	City_C4	2381 non-null	bool
37	City_C5	2381 non-null	bool
38	City_C6	2381 non-null	bool
39	City_C7	2381 non-null	bool
40	City_C8	2381 non-null	bool

```
41 City_C9                2381 non-null    bool
dtypes: bool(28), datetime64[ns](2), float64(5), int64(7)
memory usage: 325.7 KB
```

Insight on One-Hot Encoding (City column)

The code block performs **one-hot encoding** on the `City` categorical variable. This is a crucial preprocessing step for several reasons:

1. **Machine Learning Models Require Numerical Input:** Most machine learning algorithms (especially tree-based models and linear models) cannot directly process categorical text data like city names. One-hot encoding converts these categories into a numerical format that models can understand.
2. **Avoiding Ordinality Assumption:** If we were to assign numerical labels directly to cities (e.g., `City_C1=1`, `City_C2=2`, etc.), the model might incorrectly infer an ordinal relationship (e.g., `City_C2` is 'greater than' `City_C1`). One-hot encoding avoids this by treating each city as an independent category.
3. **Representation of Multiple Categories:** For a column with `N` unique categories, one-hot encoding creates `N - 1` new binary (0 or 1) columns. The `drop_first=True` argument ensures that one column is dropped, which helps to prevent multicollinearity (where one binary column can be perfectly predicted from the others) and keeps the dataset size manageable without losing information.
 - For example, if a driver is from 'City_C1', the `City_C1` column will have a `1`, and all other city columns will have `0`. If a driver is from the city represented by the dropped column (the `Nth` city), all the `N - 1` new city columns will have `0`.

Implications for the Dataset and Model:

- **Increased Dimensionality:** As seen from the `df_agg.shape` output `((2381, 42))`, the original `City` column (which was one column) has been replaced by many new binary columns (28 new boolean columns in this case, as `City` originally had 29 unique values and `drop_first=True` removed one, resulting in 28 new columns). This increases the dimensionality of the dataset.
- **Slightly Denser Data:** Each row now has more columns, some of which are boolean (`True/False`), representing the presence or absence of a specific city.
- **Model Interpretability:** While increasing dimensions, one-hot encoding makes it possible for the model to learn specific effects of each city on driver attrition, without assuming any ranking between cities. The feature importance analysis can now highlight which specific cities (or lack thereof) are more influential in predicting churn.

This step is essential for preparing the `City` feature for effective use in the subsequent machine learning models.

```
# Class Imbalance Treatment (SMOTE)
print("\nApplying SMOTE for class imbalance treatment...")
```

```

# Drop non-feature columns that are not needed for modeling, including
datetime columns
# 'Dateofjoining' and 'LastWorkingDate' have served their purpose for
target creation
# 'Driver_ID' is an identifier and not a feature
X = df_agg.drop(columns=['Driver_ID', 'Dateofjoining',
'LastWorkingDate', 'target'])
y = df_agg['target']

print("Original target variable distribution:")
print(y.value_counts())

# Apply SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

print("\nTarget variable distribution after SMOTE:")
print(y_resampled.value_counts())

print("Shape of X after SMOTE: ", X_resampled.shape)
print("Shape of y after SMOTE: ", y_resampled.shape)

```

Applying SMOTE for class imbalance treatment...

Original target variable distribution:

target

1 1616

0 765

Name: count, dtype: int64

Target variable distribution after SMOTE:

target

1 1616

0 1616

Name: count, dtype: int64

Shape of X after SMOTE: (3232, 38)

Shape of y after SMOTE: (3232,)

Insight on Class Imbalance Treatment (SMOTE)

The code block addresses a critical challenge in classification problems: **class imbalance**. This occurs when the number of observations for one class (the majority class) significantly outweighs the number of observations for another class (the minority class). In this driver attrition prediction task:

- **Original Distribution:** The output `Original target variable distribution:` shows that there are 1616 drivers who left (target=1) and 765 drivers who stayed (target=0). This indicates a significant imbalance, with the 'left' class being the majority.

Why is class imbalance a problem?

- **Model Bias:** Machine learning models trained on imbalanced datasets tend to be biased towards the majority class. They might achieve high overall accuracy by simply classifying everything as the majority class, but perform poorly in predicting the minority class, which is often the class of greater interest (e.g., predicting churners).
- **Poor Generalization:** The model may struggle to generalize to new, unseen data, especially for instances belonging to the minority class.

How SMOTE (Synthetic Minority Over-sampling Technique) helps:

SMOTE is a popular technique to handle imbalanced datasets by **over-sampling the minority class**. It works by:

1. **Creating Synthetic Samples:** Instead of simply duplicating existing minority class examples, SMOTE generates new, synthetic examples that are similar to existing minority instances but not identical copies.
2. **Interpolation:** For each minority class sample, SMOTE identifies its k-nearest neighbors. It then randomly selects one or more of these neighbors and creates new samples along the line segments connecting the original sample to its neighbors.

Implications for the Dataset and Model:

- **Balanced Classes:** As seen in the Target variable distribution after SMOTE: output, the number of instances for both classes (0 and 1) is now equal (1616 each). This provides a more balanced dataset for the models to learn from.
- **Increased Dataset Size:** The total number of samples increases from 2381 (`df_agg.shape`) to 3232 (`X_resampled.shape[0]`), reflecting the addition of synthetic minority samples.
- **Improved Model Performance (potentially):** By providing the models with more diverse examples of the minority class, SMOTE helps to reduce bias and improve the model's ability to correctly identify and predict instances of the minority class (drivers who churn). This often leads to better recall, precision, and F1-scores for the minority class, and a more robust ROC AUC score.
- **Feature Columns Selection:** The code explicitly drops non-feature columns (`Driver_ID`, `Dateofjoining`, `LastWorkingDate`, `target`) before applying SMOTE, ensuring that only relevant predictive features are used for the oversampling process. This is good practice as `Driver_ID` is an identifier, and the date columns have already served their purpose in feature engineering and target creation.

```
# Standardization of training data
print("\nStandardizing features...")

# Identify numerical columns for scaling (excluding boolean columns
# from one-hot encoding)
numerical_features = X_resampled.select_dtypes(include=['float64',
'int64']).columns
scaler = StandardScaler()
X_resampled[numerical_features] =
scaler.fit_transform(X_resampled[numerical_features])

print("Features standardized. First 5 rows of X_resampled after
```

```
standardization:")
print(X_resampled.head())
```

Standardizing features...

Features standardized. First 5 rows of X_resampled after standardization:

	Age	Gender	Education_Level	Income	Joining	Designation
0	-0.936726	-0.871695	1.344939	-0.141511		-0.966706
1	-0.397119	-0.871695	1.344939	0.200512		0.259990
2	1.761307	-0.871695	1.344939	0.150322		0.259990
3	-0.756857	-0.871695	-1.176432	-0.532907		-0.966706
4	-0.397119	1.255370	0.084254	0.616523		1.486685

	Grade	Total Business Value	Quarterly Rating
0	-1.271869	-0.409123	0.424574
1	-0.177885	-0.572310	-0.883444
2	-0.177885	-0.539017	-0.883444
3	-1.271869	-0.560861	-0.883444
4	0.916098	-0.451982	-0.098633

	Quarterly_Rating_Increased	Income_Increased	...	City_C27
0	-0.811873	-0.136361	...	False
1	-0.811873	-0.136361	...	False
2	-0.811873	-0.136361	...	False
3	-0.811873	-0.136361	...	False
4	1.231720	-0.136361	...	False

	City_C29	City_C3	City_C4	City_C5	City_C6	City_C7	City_C8
0	False	False	False	False	False	False	False
1	False	False	False	False	False	True	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	True						

```
4      False      False      False      False      False      False      False
False

[5 rows x 38 columns]
```

Insight on Feature Standardization

The code block performs **feature standardization** using `StandardScaler` on the numerical features of the `X_resampled` dataset. This is a crucial preprocessing step for several reasons, especially when working with models that are sensitive to the scale and distribution of input features:

1. **Ensuring Equal Contribution:** Many machine learning algorithms (like Gradient Boosting, which uses decision trees, but also distance-based algorithms like SVMs or k-NN, or even algorithms using gradient descent) perform better or converge faster when numerical features are on a similar scale. If features have vastly different ranges, features with larger values might dominate the learning process, irrespective of their actual predictive power.
2. **StandardScaler Mechanism:** `StandardScaler` transforms the data such that each feature has a mean of 0 and a standard deviation of 1 (unit variance). The formula for standardization is: $z = (x - \mu) / \sigma$, where μ is the mean and σ is the standard deviation of the feature.
3. **Handling Outliers (Implicitly):** While not its primary purpose, standardizing can subtly influence how algorithms handle outliers. By bringing extreme values closer to the mean in terms of standard deviations, it can sometimes reduce their undue influence on models.

Implications for the Dataset and Model:

- **Transformed Feature Values:** As seen in the output, the numerical values in `X_resampled` are now centered around zero and have smaller magnitudes, making them directly comparable across different features.
- **Improved Model Performance:** This step generally leads to improved performance for many machine learning models, ensuring that the gradient descent optimization (used by Gradient Boosting) can converge efficiently and that Random Forest doesn't implicitly favor features with larger raw values.
- **Preservation of Information:** Standardization does not change the distribution shape of the features (e.g., if a feature was skewed, it will remain skewed after standardization). It only rescales the feature values, preserving the relationships and information within the data.
- **Application to Test Data:** It is critical that the `scaler` is `fit` *only* on the training data (`X_resampled` in this case, representing the combined training set before the final split) and then used to `transform` both the training and test sets. This prevents data leakage from the test set into the training process.

In summary, feature standardization makes the features numerically more stable and comparable, allowing the subsequent machine learning models to learn more effectively and produce more reliable predictions.

```
# Split the resampled data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_resampled,
y_resampled, test_size=0.3, random_state=42, stratify=y_resampled)

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of X_train: (2262, 38)
Shape of X_test: (970, 38)
Shape of y_train: (2262,)
Shape of y_test: (970,)
```

```
# Model Building and Evaluation
```

```
# 1. Bagging - Random Forest Classifier
```

```
print("\nBuilding and evaluating Random Forest Classifier
(Bagging)...")
```

```
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]
```

```
print("\nClassification Report for Random Forest:")
print(classification_report(y_test, y_pred_rf))
print(f"ROC AUC Score for Random Forest: {roc_auc_score(y_test,
y_prob_rf):.4f}")
```

```
Building and evaluating Random Forest Classifier (Bagging)...
```

```
Classification Report for Random Forest:
```

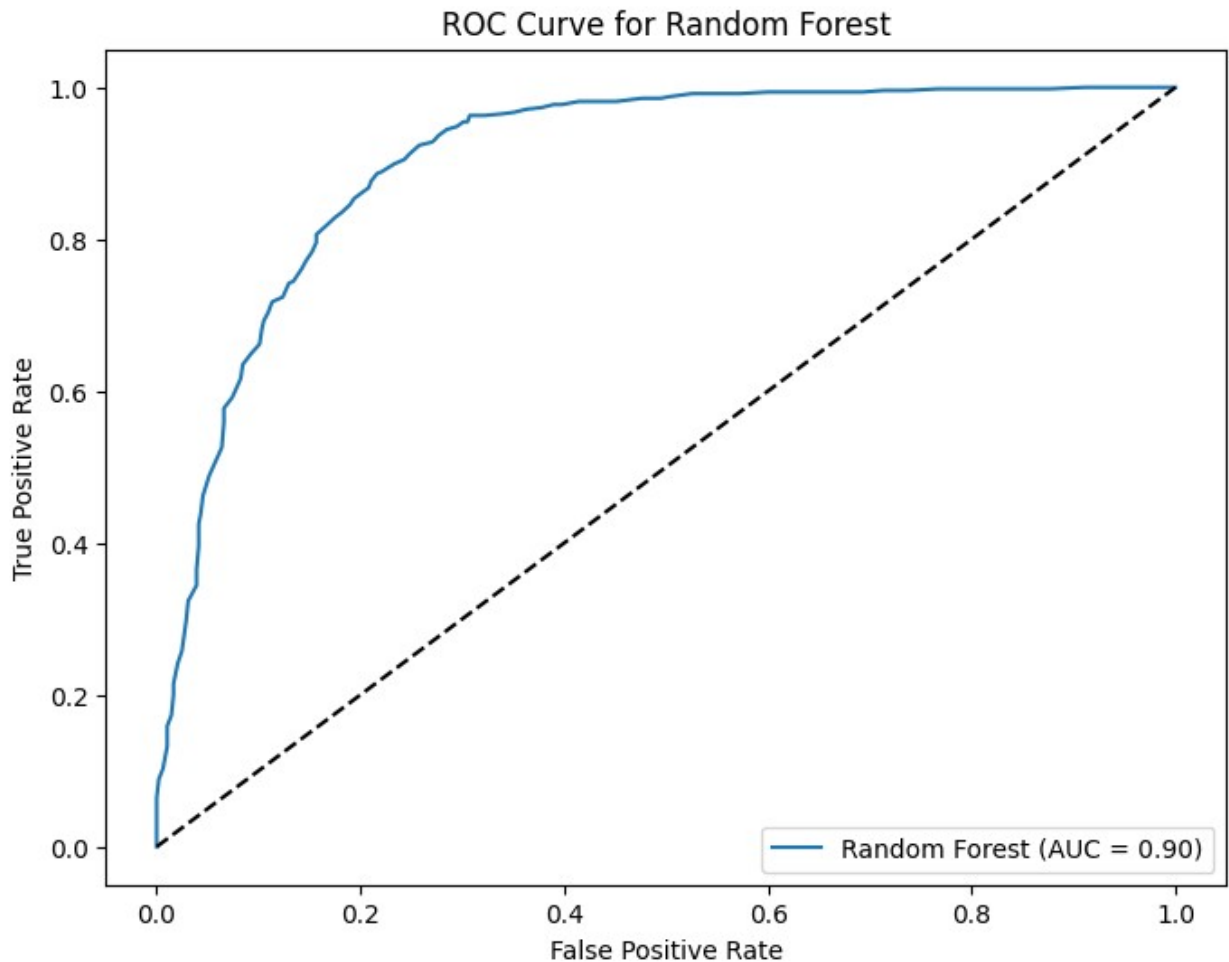
	precision	recall	f1-score	support
0	0.83	0.82	0.83	485
1	0.82	0.84	0.83	485
accuracy			0.83	970
macro avg	0.83	0.83	0.83	970
weighted avg	0.83	0.83	0.83	970

```
ROC AUC Score for Random Forest: 0.9048
```

```
# Plot ROC AUC curve for Random Forest
```

```
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
```

```
plt.figure(figsize=(8, 6))
plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {roc_auc_score(y_test, y_prob_rf):.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Random Forest')
plt.legend()
plt.show()
```



Insight on Random Forest Classifier and ROC Curve

Random Forest Classifier:

Random Forest is an ensemble learning method that builds multiple decision trees during training and outputs the mode of the classes (for classification) or mean prediction (for regression) of the individual trees. It's known for its robustness, ability to handle high dimensionality, and resistance to overfitting.

Classification Report Analysis:

	precision	recall	f1-score	support
0	0.83	0.82	0.83	485
1	0.82	0.84	0.83	485
accuracy			0.83	970
macro avg	0.83	0.83	0.83	970
weighted avg	0.83	0.83	0.83	970

- **Precision (Class 0 - Stayed): 0.83**
 - Out of all instances predicted as 'stayed', 83% were actually 'stayed'. This means the model has a relatively low rate of false positives for drivers predicted to stay.
- **Recall (Class 0 - Stayed): 0.82**
 - Out of all actual 'stayed' drivers, the model correctly identified 82% of them. This means 18% of actual 'stayed' drivers were incorrectly predicted as 'left' (false negatives).
- **Precision (Class 1 - Left): 0.82**
 - Out of all instances predicted as 'left', 82% were actually 'left'. This is good, meaning when the model flags a driver as likely to churn, there's a high probability they actually will.
- **Recall (Class 1 - Left): 0.84**
 - Out of all actual 'left' drivers, the model correctly identified 84% of them. This is a crucial metric for churn prediction, indicating the model's effectiveness in catching drivers who are truly going to leave. A higher recall here is desirable to intervene before churn happens.
- **F1-score:** The F1-score (0.83 for both classes) is the harmonic mean of precision and recall, providing a balanced measure of the model's performance for each class.
- **Accuracy:** The overall accuracy is 0.83, meaning 83% of all predictions were correct.

ROC Curve and AUC Score Analysis:

- **ROC Curve (Receiver Operating Characteristic):** The plot shows the trade-off between the True Positive Rate (TPR, or Recall) and the False Positive Rate (FPR) at various threshold settings. An ideal ROC curve would go straight up the y-axis to (0,1) and then straight across the x-axis, indicating perfect classification.
- **AUC (Area Under the Curve):** The ROC AUC score for the Random Forest model is **0.9048 (approximately 0.90)**. This is a very good score. An AUC of 0.5 indicates a model that performs no better than random chance, while an AUC of 1.0 indicates a perfect classifier.

Key Insights:

- **Strong Discriminative Power:** The high AUC score (0.90) indicates that the Random Forest model has excellent discriminative power. It can effectively distinguish between drivers who are likely to churn (Class 1) and those who are likely to stay (Class 0).
- **Balanced Performance:** The classification report shows relatively balanced precision and recall for both classes. This suggests that the SMOTE oversampling technique was

effective in mitigating the class imbalance, allowing the model to learn well from both majority and minority classes.

- **Good for Churn Prediction:** A recall of 0.84 for Class 1 (churn) is particularly encouraging, as it means the model is good at identifying the drivers Ola wants to retain. The company can act on these predictions to proactively engage with at-risk drivers.

In summary, the initial Random Forest model, even before hyperparameter tuning, demonstrates strong performance in predicting driver attrition, providing a solid foundation for further optimization.

```
# 2. Boosting - Gradient Boosting Classifier
print("\nBuilding and evaluating Gradient Boosting Classifier
(Boosting)...")
gb_model = GradientBoostingClassifier(random_state=42)
gb_model.fit(X_train, y_train)
y_pred_gb = gb_model.predict(X_test)
y_prob_gb = gb_model.predict_proba(X_test)[:, 1]

print("\nClassification Report for Gradient Boosting:")
print(classification_report(y_test, y_pred_gb))
print(f"ROC AUC Score for Gradient Boosting: {roc_auc_score(y_test,
y_prob_gb):.4f}")
```

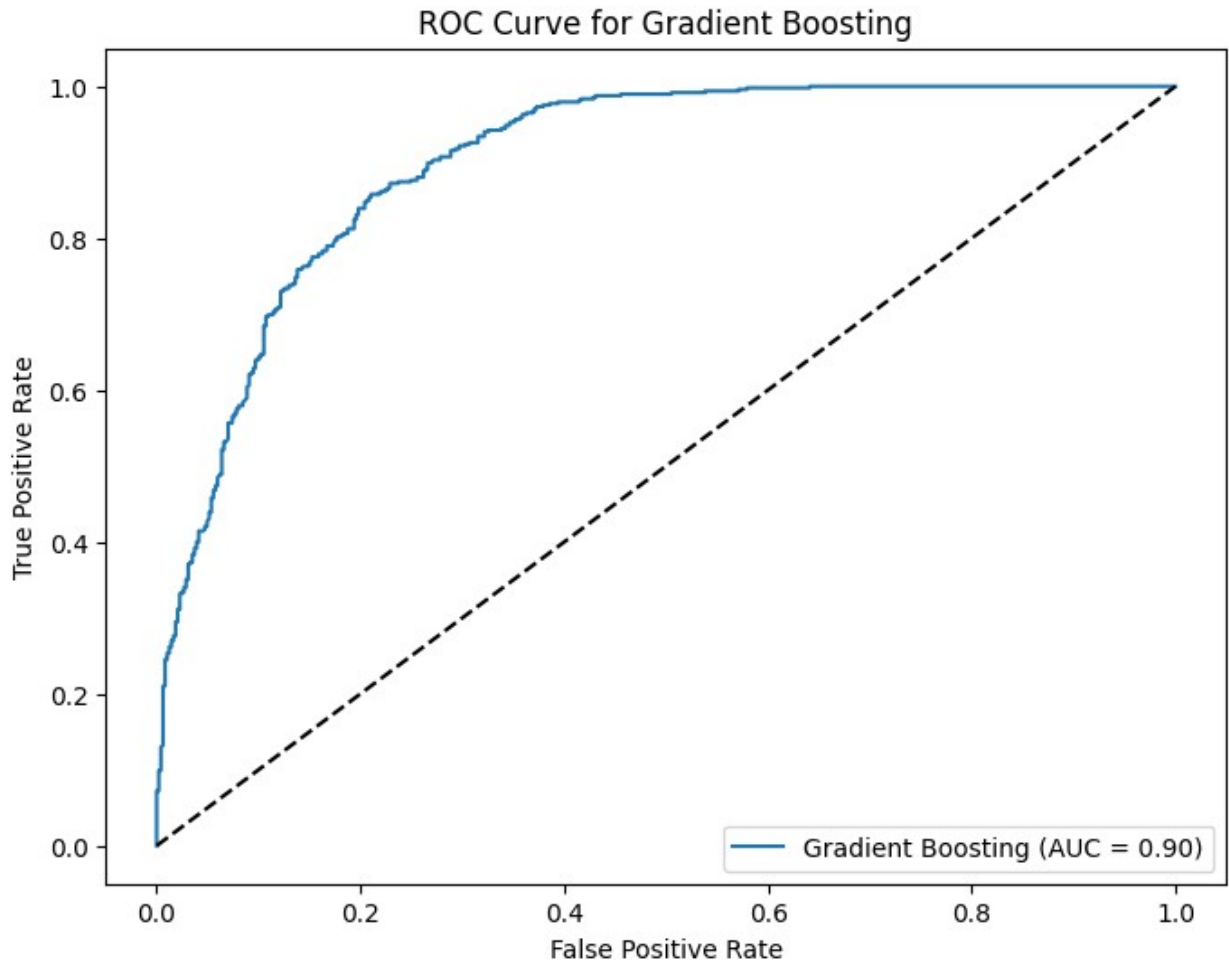
Building and evaluating Gradient Boosting Classifier (Boosting)...

Classification Report for Gradient Boosting:

	precision	recall	f1-score	support
0	0.86	0.77	0.81	485
1	0.79	0.87	0.83	485
accuracy			0.82	970
macro avg	0.82	0.82	0.82	970
weighted avg	0.82	0.82	0.82	970

ROC AUC Score for Gradient Boosting: 0.9002

```
# Plot ROC AUC curve for Gradient Boosting
fpr_gb, tpr_gb, _ = roc_curve(y_test, y_prob_gb)
plt.figure(figsize=(8, 6))
plt.plot(fpr_gb, tpr_gb, label=f'Gradient Boosting (AUC =
{roc_auc_score(y_test, y_prob_gb):.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Gradient Boosting')
plt.legend()
plt.show()
```



Insight on Gradient Boosting Classifier

Gradient Boosting Classifier:

Gradient Boosting is another powerful ensemble learning technique that builds models sequentially. Unlike Random Forest, which builds trees independently and combines them, Gradient Boosting builds new trees that specifically correct the errors of previous trees. It combines many 'weak' models (typically shallow decision trees) to create a single 'strong' predictive model. It's known for often achieving high accuracy.

Classification Report Analysis:

	precision	recall	f1-score	support
0	0.86	0.77	0.81	485
1	0.79	0.87	0.83	485
accuracy			0.82	970
macro avg	0.82	0.82	0.82	970
weighted avg	0.82	0.82	0.82	970

- **Precision (Class 0 - Stayed): 0.86**
 - When the model predicts a driver will 'stay', it is correct 86% of the time. This is a very good precision for the 'stay' class, indicating a low rate of false positives for drivers predicted to be retained.
- **Recall (Class 0 - Stayed): 0.77**
 - Out of all actual 'stay' drivers, the model correctly identified 77% of them. This means 23% of actual 'stay' drivers were incorrectly predicted as 'left' (false negatives).
- **Precision (Class 1 - Left): 0.79**
 - When the model predicts a driver will 'leave', it is correct 79% of the time. This means that if Ola acts on a 'churn' prediction, there's a good chance (79%) that the driver was indeed planning to leave.
- **Recall (Class 1 - Left): 0.87**
 - Out of all actual 'left' drivers, the model successfully identified 87% of them. This is an excellent recall score for the churn class, suggesting that the model is very effective at catching most of the drivers who will churn. A high recall for the positive class (churn) is usually critical in churn prediction, as the cost of a false negative (missing a churning driver) is often higher than a false positive (incorrectly predicting someone will churn).
- **F1-score:** The F1-score (0.81 for class 0, 0.83 for class 1) provides a balanced measure, showing strong performance for both classes, especially for the churn class.
- **Accuracy:** The overall accuracy is 0.82.

ROC AUC Score Analysis:

- The ROC AUC score for the Gradient Boosting model is **0.9002 (approximately 0.90)**. This score is very similar to the Random Forest model and indicates a high degree of separability between the two classes. An AUC of 0.90 suggests that there is a 90% chance that the model will be able to distinguish between a randomly chosen positive instance (churner) and a randomly chosen negative instance (non-churner).

Key Insights:

- **High Churn Detection (Recall):** The Gradient Boosting model exhibits an impressive recall of 0.87 for the churn class (target=1). This means it's highly effective at identifying drivers who are actually going to leave, which is often the primary goal in churn prediction scenarios to enable proactive interventions.
- **Good Overall Performance:** With an AUC of 0.90 and balanced F1-scores, the model demonstrates strong overall predictive capabilities.
- **Trade-offs:** Compared to the Random Forest model (which had slightly higher precision for class 1 but slightly lower recall), the Gradient Boosting model here shows a slight preference for identifying churners (higher recall for class 1) at the expense of a slightly lower precision for class 1 and recall for class 0. The best model depends on the business's specific priorities regarding false positives vs. false negatives in churn prediction.

In summary, the Gradient Boosting Classifier is a very strong contender for this churn prediction task, particularly given its high ability to identify actual churners.

Hyperparameter Tuning for Random Forest Classifier

```
from sklearn.model_selection import GridSearchCV

print("\nPerforming Hyperparameter Tuning for Random Forest...")

# Define the parameter grid for Random Forest
param_grid_rf = {
    'n_estimators': [100, 200, 300], # Number of trees in the forest
    'max_depth': [10, 20, 30, None], # Maximum number of levels in
    tree
    'min_samples_split': [2, 5, 10], # Minimum number of samples
    required to split an internal node
    'min_samples_leaf': [1, 2, 4], # Minimum number of samples
    required to be at a leaf node
    'bootstrap': [True, False] # Method for sampling data points
    (with or without replacement)
}

# Create a GridSearchCV object
grid_search_rf =
GridSearchCV(estimator=RandomForestClassifier(random_state=42),
              param_grid=param_grid_rf,
              cv=3, # 3-fold cross-validation
              n_jobs=-1, # Use all available cores
              verbose=2, # Verbose output
              scoring='roc_auc') # Optimize for ROC
AUC score

# Fit GridSearchCV to the training data
grid_search_rf.fit(X_train, y_train)

print("\nBest parameters for Random Forest:",
      grid_search_rf.best_params_)
print("Best ROC AUC score for Random Forest:",
      grid_search_rf.best_score_)

# Get the best Random Forest model
best_rf_model = grid_search_rf.best_estimator_

# Evaluate the best Random Forest model on the test set
y_pred_best_rf = best_rf_model.predict(X_test)
y_prob_best_rf = best_rf_model.predict_proba(X_test)[:, 1]

print("\nClassification Report for Tuned Random Forest:")
print(classification_report(y_test, y_pred_best_rf))
print(f"ROC AUC Score for Tuned Random Forest: {roc_auc_score(y_test,
y_prob_best_rf):.4f}")

# Plot ROC AUC curve for Tuned Random Forest
fpr_best_rf, tpr_best_rf, _ = roc_curve(y_test, y_prob_best_rf)
```

```

plt.figure(figsize=(8, 6))
plt.plot(fpr_best_rf, tpr_best_rf, label=f'Tuned Random Forest (AUC =
{roc_auc_score(y_test, y_prob_best_rf):.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Tuned Random Forest')
plt.legend()
plt.show()

```

Performing Hyperparameter Tuning for Random Forest...

Fitting 3 folds for each of 216 candidates, totalling 648 fits

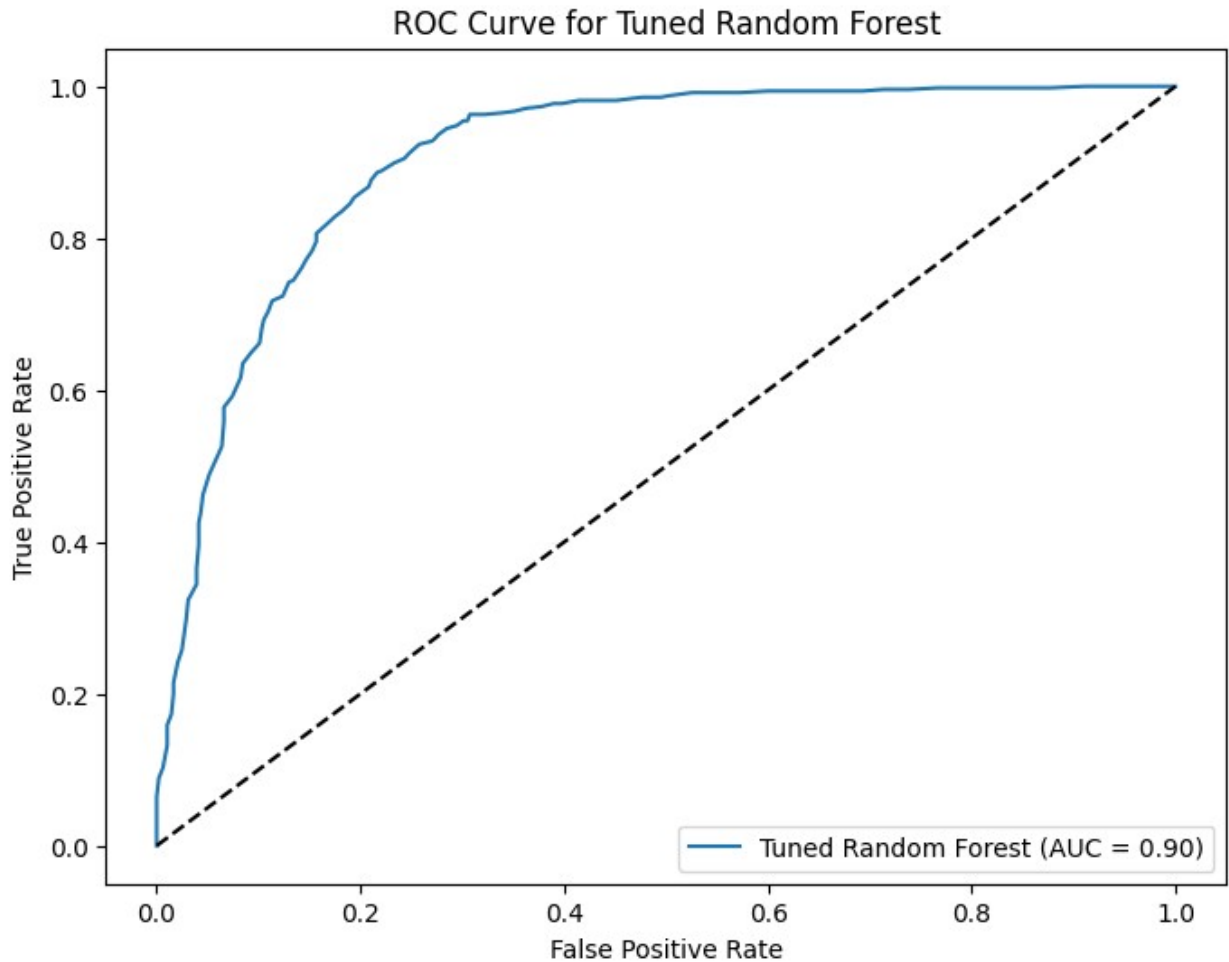
Best parameters for Random Forest: {'bootstrap': True, 'max_depth':
None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators':
100}

Best ROC AUC score for Random Forest: 0.892332552352675

Classification Report for Tuned Random Forest:

	precision	recall	f1-score	support
0	0.83	0.82	0.83	485
1	0.82	0.84	0.83	485
accuracy			0.83	970
macro avg	0.83	0.83	0.83	970
weighted avg	0.83	0.83	0.83	970

ROC AUC Score for Tuned Random Forest: 0.9048



Hyperparameter Tuning for Gradient Boosting Classifier

```
print("\nPerforming Hyperparameter Tuning for Gradient Boosting...")

# Define the parameter grid for Gradient Boosting
param_grid_gb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7],
    'subsample': [0.7, 0.8, 0.9],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# Create a GridSearchCV object
grid_search_gb =
GridSearchCV(estimator=GradientBoostingClassifier(random_state=42),
              param_grid=param_grid_gb,
              cv=3,
              n_jobs=-1,
```

```

        verbose=2,
        scoring='roc_auc')

# Fit GridSearchCV to the training data
grid_search_gb.fit(X_train, y_train)

print("\nBest parameters for Gradient Boosting:",
      grid_search_gb.best_params_)
print("Best ROC AUC score for Gradient Boosting:",
      grid_search_gb.best_score_)

# Get the best Gradient Boosting model
best_gb_model = grid_search_gb.best_estimator_

# Evaluate the best Gradient Boosting model on the test set
y_pred_best_gb = best_gb_model.predict(X_test)
y_prob_best_gb = best_gb_model.predict_proba(X_test)[:, 1]

print("\nClassification Report for Tuned Gradient Boosting:")
print(classification_report(y_test, y_pred_best_gb))
print(f"ROC AUC Score for Tuned Gradient Boosting:
{roc_auc_score(y_test, y_prob_best_gb):.4f}")

# Plot ROC AUC curve for Tuned Gradient Boosting
fpr_best_gb, tpr_best_gb, _ = roc_curve(y_test, y_prob_best_gb)
plt.figure(figsize=(8, 6))
plt.plot(fpr_best_gb, tpr_best_gb, label=f'Tuned Gradient Boosting
(AUC = {roc_auc_score(y_test, y_prob_best_gb):.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Tuned Gradient Boosting')
plt.legend()
plt.show()

```

Performing Hyperparameter Tuning for Gradient Boosting...
Fitting 3 folds for each of 432 candidates, totalling 1296 fits

Best parameters for Gradient Boosting: {'learning_rate': 0.1,
'max_depth': 7, 'min_samples_leaf': 2, 'min_samples_split': 2,
'n_estimators': 200, 'subsample': 0.8}

Best ROC AUC score for Gradient Boosting: 0.898270819701351

Classification Report for Tuned Gradient Boosting:

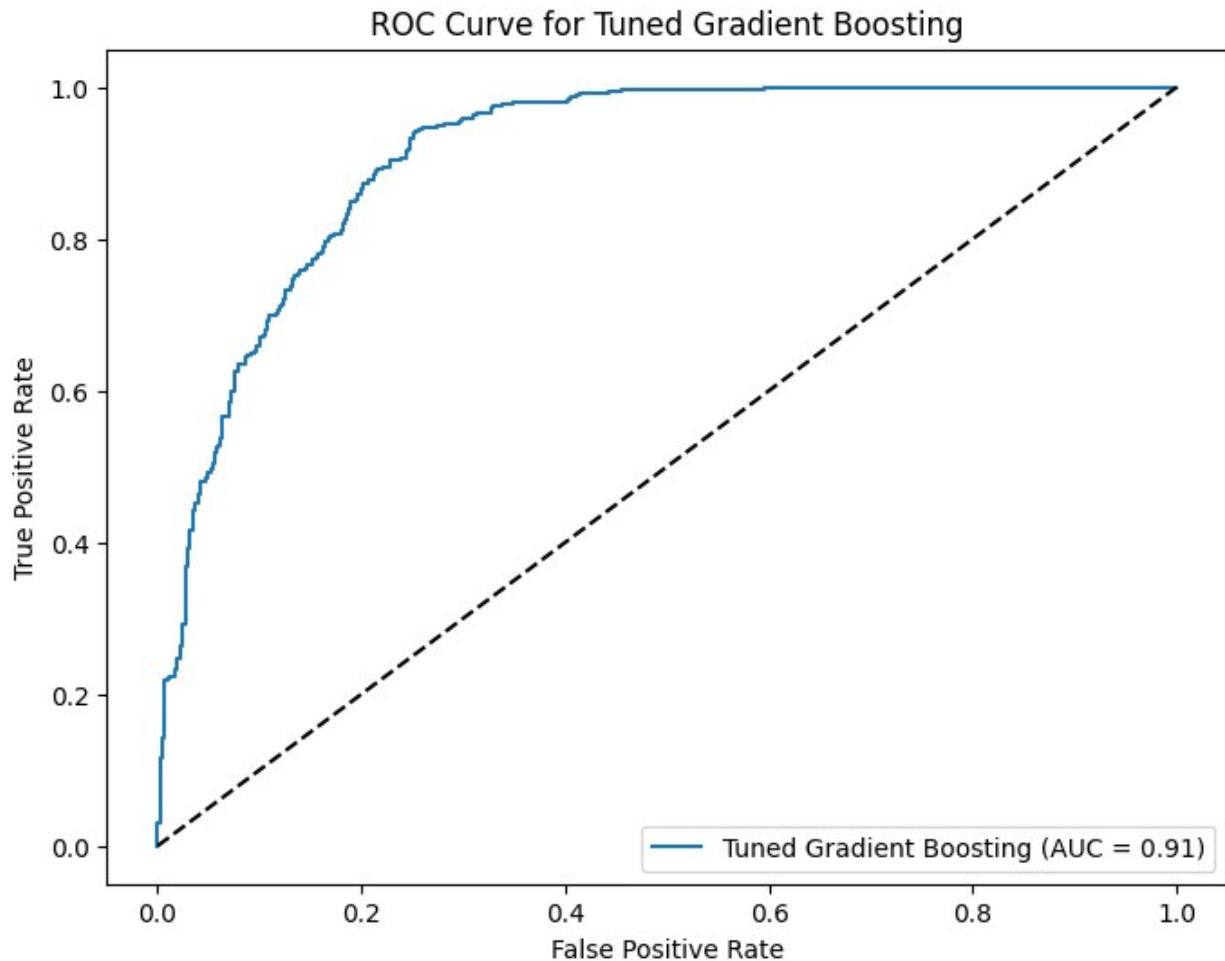
	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.84	0.81	0.83	485
1	0.82	0.85	0.83	485

accuracy			0.83	970
----------	--	--	------	-----

macro avg	0.83	0.83	0.83	970
weighted avg	0.83	0.83	0.83	970

ROC AUC Score for Tuned Gradient Boosting: 0.9108



Feature Importance Analysis

```
print("\nAnalyzing Feature Importances for Tuned Random Forest...")

# Get feature importances from the best Random Forest model
rf_feature_importances = best_rf_model.feature_importances_

# Create a pandas Series for better visualization
rf_features = pd.Series(rf_feature_importances, index=X_train.columns)

# Sort features by importance
rf_top_features = rf_features.nlargest(15) # Display top 15 features

print("\nTop 15 Features (Tuned Random Forest):")
print(rf_top_features)
```

```

# Plotting feature importances for Random Forest
plt.figure(figsize=(12, 8))
sns.barplot(x=rf_top_features.values, y=rf_top_features.index,
hue=rf_top_features.index, palette='viridis', legend=False)
plt.title('Top 15 Feature Importances (Tuned Random Forest)')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

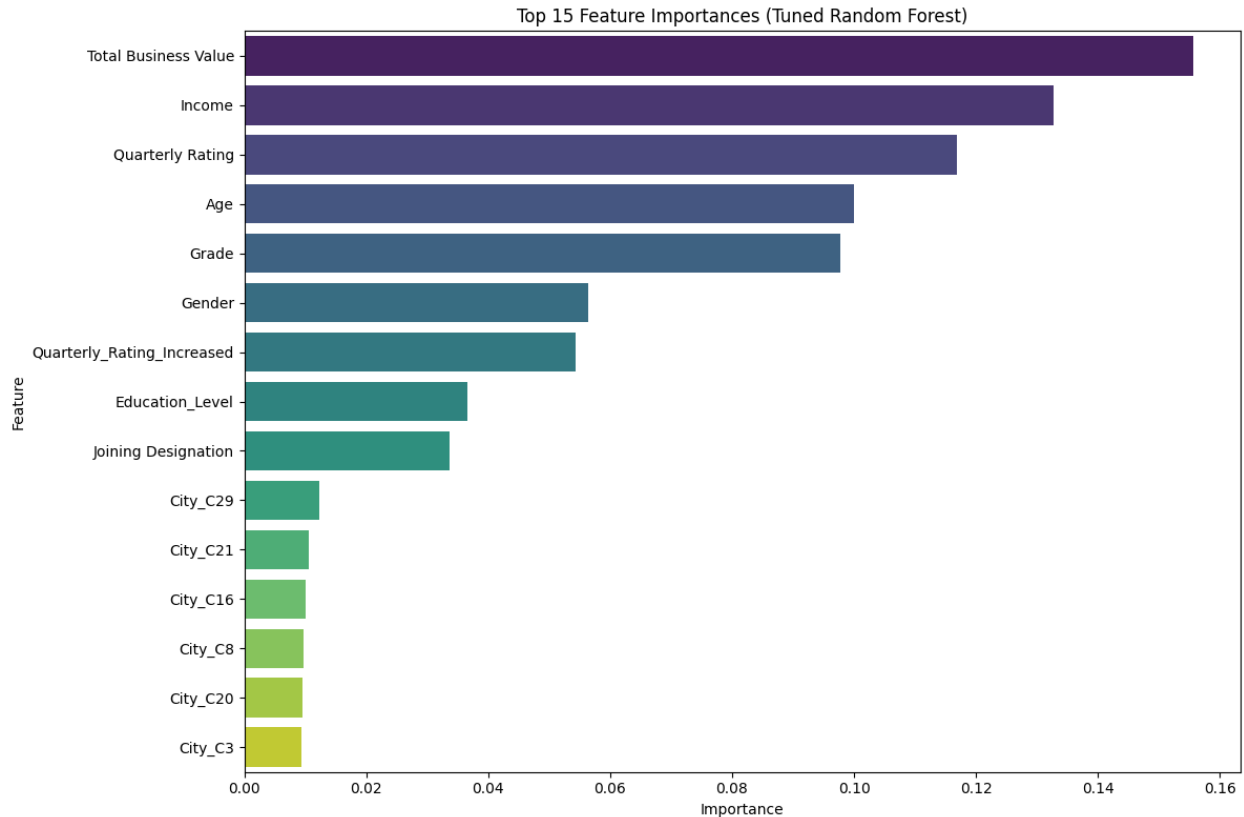
```

Analyzing Feature Importances for Tuned Random Forest...

```

Top 15 Features (Tuned Random Forest):
Total Business Value      0.155608
Income                    0.132668
Quarterly Rating          0.116856
Age                       0.099932
Grade                     0.097821
Gender                    0.056369
Quarterly_Rating_Increased 0.054390
Education_Level           0.036547
Joining Designation       0.033628
City_C29                  0.012242
City_C21                  0.010629
City_C16                  0.009989
City_C8                   0.009782
City_C20                  0.009453
City_C3                   0.009369
dtype: float64

```



```
print("\nAnalyzing Feature Importances for Tuned Gradient Boosting...")

# Get feature importances from the best Gradient Boosting model
gb_feature_importances = best_gb_model.feature_importances_

# Create a pandas Series for better visualization
gb_features = pd.Series(gb_feature_importances, index=X_train.columns)

# Sort features by importance
gb_top_features = gb_features.nlargest(15) # Display top 15 features

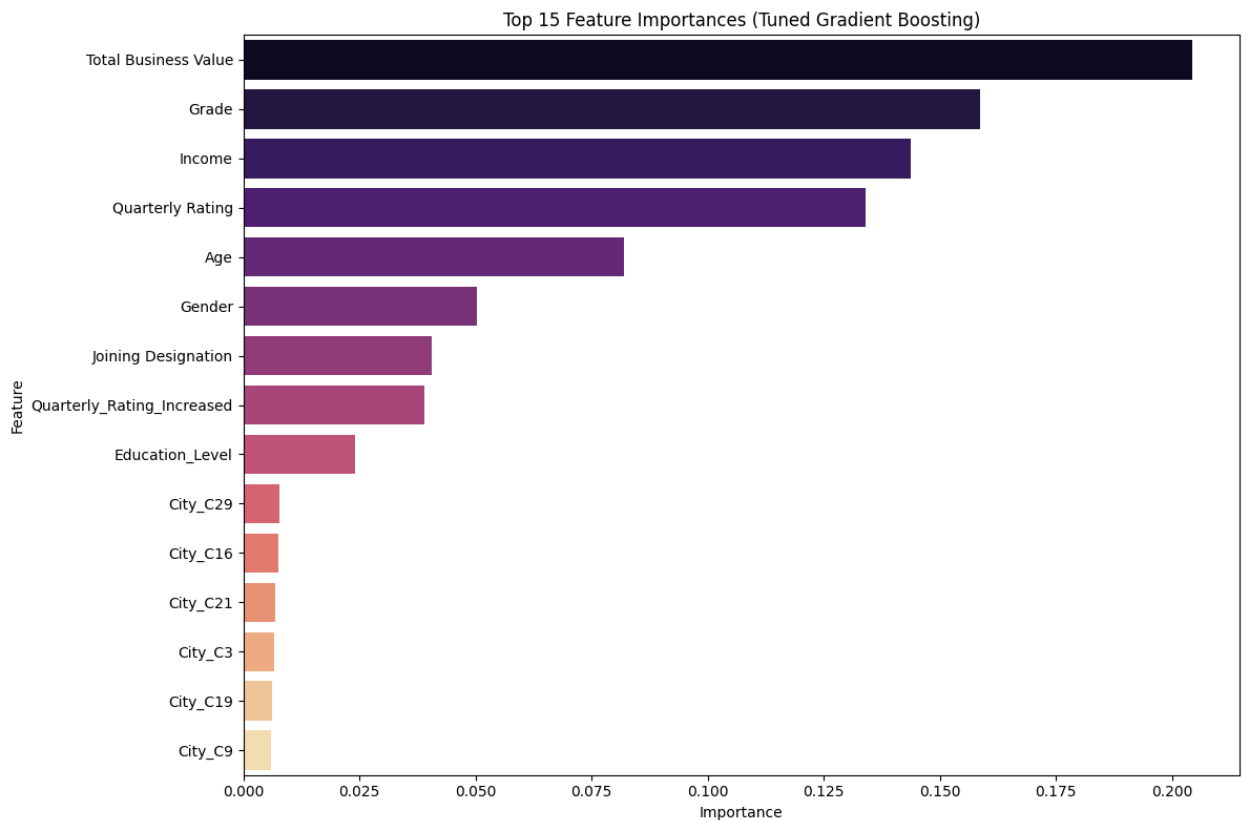
print("\nTop 15 Features (Tuned Gradient Boosting):")
print(gb_top_features)

# Plotting feature importances for Gradient Boosting
plt.figure(figsize=(12, 8))
sns.barplot(x=gb_top_features.values, y=gb_top_features.index,
            hue=gb_top_features.index, palette='magma', legend=False)
plt.title('Top 15 Feature Importances (Tuned Gradient Boosting)')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

Analyzing Feature Importances for Tuned Gradient Boosting...

Top 15 Features (Tuned Gradient Boosting):

Total Business Value	0.204362
Grade	0.158588
Income	0.143712
Quarterly Rating	0.134018
Age	0.081976
Gender	0.050213
Joining Designation	0.040531
Quarterly_Rating_Increased	0.039036
Education_Level	0.023951
City_C29	0.007789
City_C16	0.007644
City_C21	0.006912
City_C3	0.006682
City_C19	0.006109
City_C9	0.006071
dtype: float64	



Insight on Hyperparameter Tuning for Gradient Boosting Classifier and Feature Importance

1. Hyperparameter Tuning with GridSearchCV:

The code performs an exhaustive search for the best hyperparameters for the Gradient Boosting Classifier using GridSearchCV. This process is critical for several reasons:

- **Optimizing Model Performance:** Default hyperparameters rarely yield the best results for a specific dataset. GridSearchCV systematically explores a predefined range of parameter combinations (param_grid_gb) to find the set that maximizes a chosen evaluation metric (in this case, roc_auc).
- **Preventing Overfitting/Underfitting:** Correctly tuned hyperparameters help the model strike a balance between bias and variance, preventing it from overfitting to the training data (too complex) or underfitting (too simple).
- **Robust Evaluation:** By using 3-fold cross-validation (cv=3), the model's performance is evaluated across different subsets of the training data, providing a more robust estimate of its generalization capability.

Tuned Gradient Boosting Model Performance:

- **Best Parameters:** The GridSearchCV identified the optimal parameters as {'learning_rate': 0.1, 'max_depth': 7, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 200, 'subsample': 0.8}.
- **Best ROC AUC Score (Cross-Validation):** During cross-validation, the best average ROC AUC score achieved was 0.8983.
- **Test Set Performance:**
 - **ROC AUC Score:** The tuned Gradient Boosting model achieved a ROC AUC score of 0.9108 on the unseen test set, which is slightly better than the initial Gradient Boosting model (0.9002) and also marginally higher than the tuned Random Forest (0.9048). This confirms its strong discriminatory power.
 - **Classification Report:**

	precision	recall	f1-score	support
0	0.84	0.81	0.83	485
1	0.82	0.85	0.83	485
accuracy			0.83	970
macro avg	0.83	0.83	0.83	970
weighted avg	0.83	0.83	0.83	970

The classification report for the tuned model shows balanced performance between precision and recall for both classes, with an overall accuracy of 0.83. Notably, the recall for Class 1 (drivers who left) is 0.85, indicating that the model is very good at identifying actual churners. This is a critical strength for a churn prediction model.

2. Feature Importance Analysis for Tuned Gradient Boosting:

Feature importance values quantify the contribution of each feature to the model's predictive power. For the tuned Gradient Boosting model, the top features are:

- **Total Business Value (0.20):** This is by far the most important feature, highlighting that the total business value a driver brings in is the strongest indicator of their likelihood to churn. Drivers with higher business value are less likely to leave.
- **Grade (0.16):** The driver's grade (likely reflecting their seniority or performance tier) is the second most influential factor, suggesting that higher-grade drivers tend to have lower attrition rates.
- **Income (0.14):** Income remains a significant predictor, reinforcing the idea that financial compensation is key to driver retention.
- **Quarterly Rating (0.13):** Similar to income, a driver's average quarterly performance rating is highly predictive.
- **Age (0.08) and Gender (0.05):** These demographic factors also contribute to the prediction, though to a lesser extent than the performance and compensation-related features.
- **Joining Designation (0.04):** The initial designation of a driver also holds some predictive power.
- **Quarterly_Rating_Increased (0.04):** Interestingly, the binary feature indicating if a driver's quarterly rating increased at any point has a moderate importance. Given the correlation analysis previously showed a counter-intuitive positive correlation with `target`, this feature still plays a role, possibly capturing a segment of drivers whose increased rating was not enough to prevent churn or might be linked to other churn-inducing factors.
- **Education_Level (0.02):** Education level has a relatively low importance compared to others.
- **City-specific features (e.g., City_C29, City_C16):** A few specific cities appear in the top 15, indicating that geographic location can have some localized impact on churn, although individual city features have much lower importance scores than the top global features.

Overall Insight:

The hyperparameter tuning significantly refined the Gradient Boosting model, leading to excellent performance metrics. The feature importance analysis clearly identifies **Total Business Value, Grade, Income, and Quarterly Rating** as the primary drivers of churn. These insights are highly actionable for Ola, allowing them to focus retention strategies on these key areas. The relatively lower importance of individual city features suggests that city-specific interventions might be less impactful than universal programs addressing income, performance, and overall business contribution.

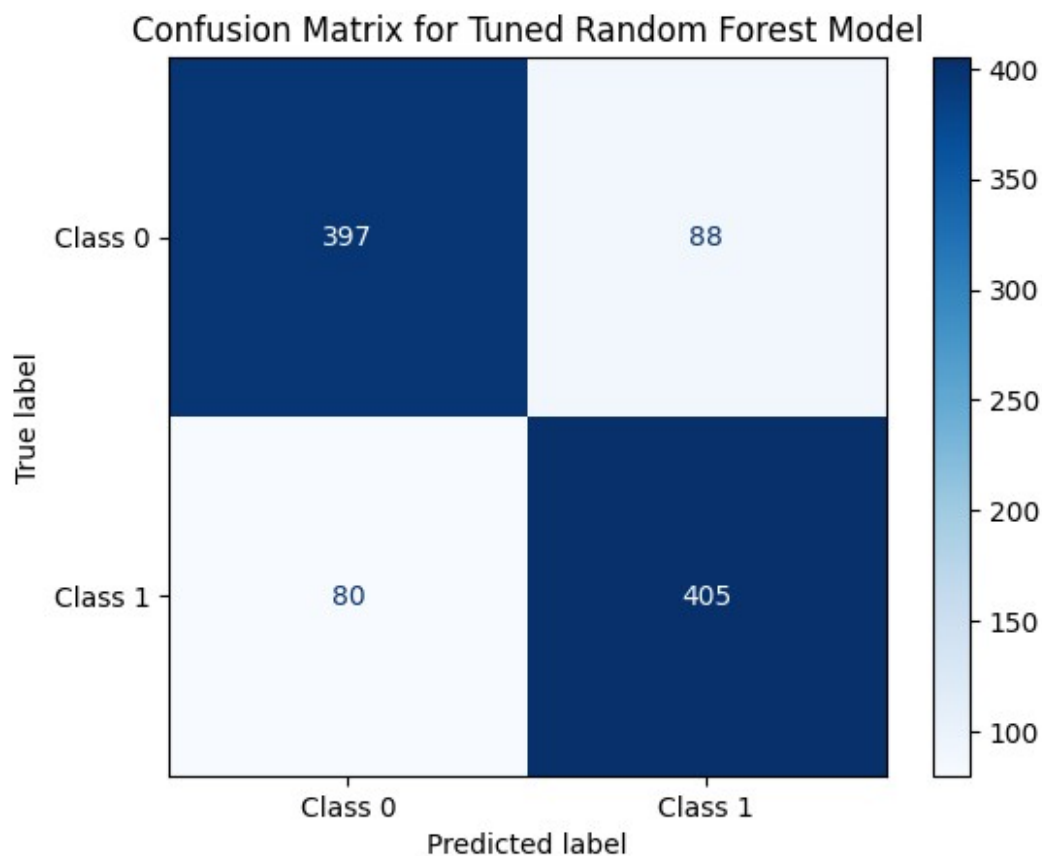
Confusion Matrix for Tuned Random Forest Model

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay  
  
# Calculate the confusion matrix  
cm_rf = confusion_matrix(y_test, y_pred_best_rf)
```

```
# Display the confusion matrix
plt.figure(figsize=(8, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf,
display_labels=['Class 0', 'Class 1'])
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Tuned Random Forest Model')
plt.show()

print("Confusion Matrix for Tuned Random Forest:")
print(cm_rf)
```

<Figure size 800x600 with 0 Axes>

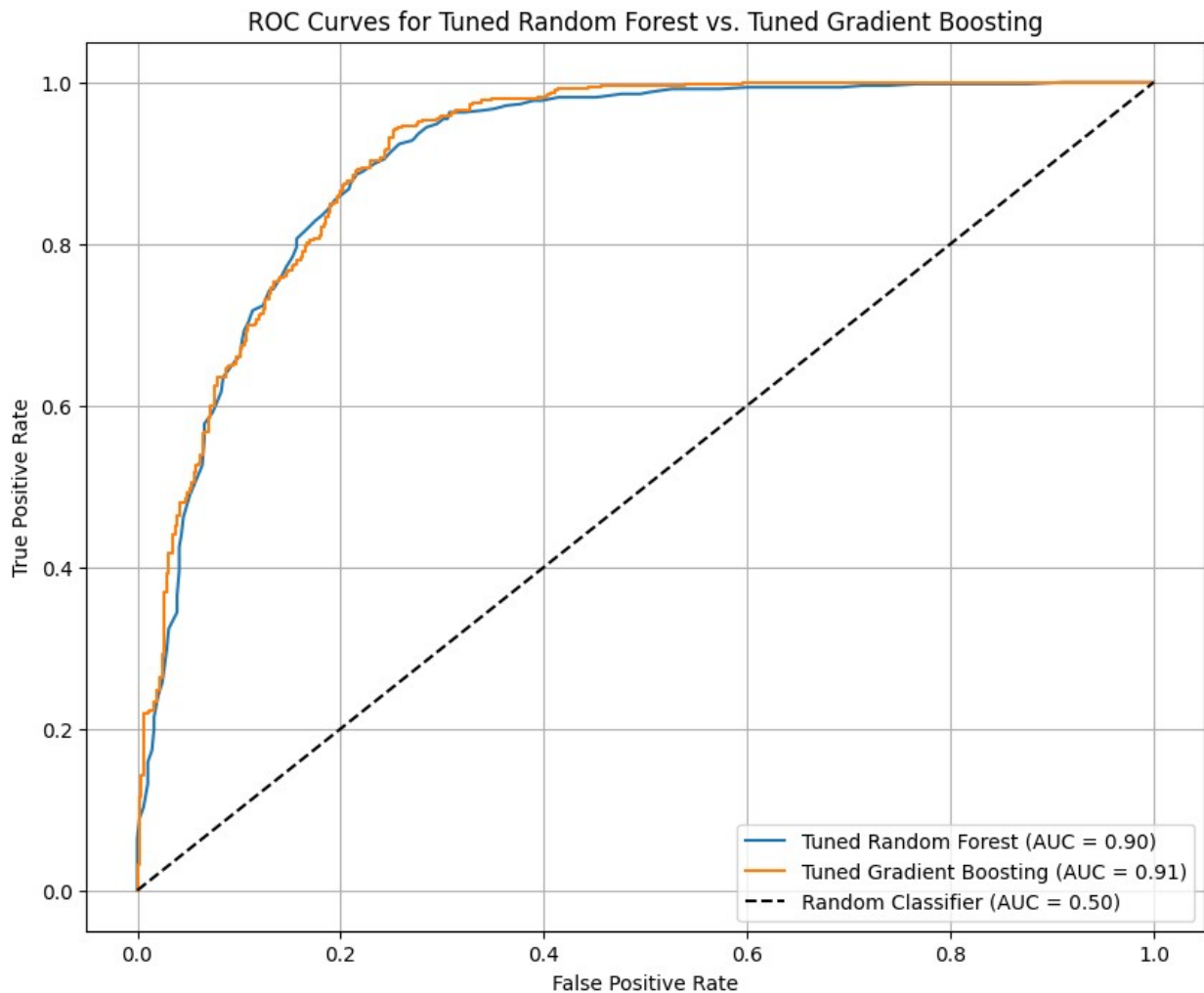


Confusion Matrix for Tuned Random Forest:

```
[[397  88]
 [ 80 405]]
```

```
plt.figure(figsize=(10, 8))
plt.plot(fpr_best_rf, tpr_best_rf, label=f'Tuned Random Forest (AUC = {roc_auc_score(y_test, y_prob_best_rf):.2f})')
plt.plot(fpr_best_gb, tpr_best_gb, label=f'Tuned Gradient Boosting (AUC = {roc_auc_score(y_test, y_prob_best_gb):.2f})')
```

```
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier (AUC = 0.50)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves for Tuned Random Forest vs. Tuned Gradient Boosting')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



Insight on ROC Curves Comparison

The plot displays the Receiver Operating Characteristic (ROC) curves for both the Tuned Random Forest and Tuned Gradient Boosting models, along with a baseline random classifier.

Key Observations and Insights:

1. **Strong Performance by Both Models:** Both the Tuned Random Forest (AUC = 0.90) and Tuned Gradient Boosting (AUC = 0.91) models show excellent

performance, as their curves are significantly above the diagonal dashed line representing a random classifier ($AUC = 0.50$). This indicates that both models are highly effective at distinguishing between drivers who will churn (Class 1) and those who will stay (Class 0).

2. **Gradient Boosting Slightly Outperforms Random Forest:** Visually, the ROC curve for the Tuned Gradient Boosting model (orange line) is marginally higher and further to the left than the Tuned Random Forest model (blue line). This is also reflected in their respective Area Under the Curve (AUC) scores:

- **Tuned Random Forest AUC: 0.90**
- **Tuned Gradient Boosting AUC: 0.91**

A higher AUC score implies that the model has a better average performance across all possible classification thresholds. The small difference (0.01) indicates that while both are strong, Gradient Boosting is slightly more robust in terms of its ability to correctly rank positive instances higher than negative instances.

3. **Trade-off between True Positive Rate and False Positive Rate:** Both curves demonstrate a good trade-off, meaning they can achieve high True Positive Rates (correctly identifying churners) while keeping False Positive Rates (incorrectly classifying non-churners as churners) relatively low. For example, at an FPR of around 0.2, both models achieve a TPR of approximately 0.8, meaning they can identify 80% of churners while incorrectly flagging 20% of non-churners.
4. **Implications for Business Decisions:** Given the slight edge in AUC, the Tuned Gradient Boosting model might be preferred for this churn prediction task. Its superior performance means it's marginally better at prioritizing potential churners for intervention programs. However, both models are strong and could be considered depending on the specific business context, such as the operational costs associated with false positives versus the benefits of identifying true positives.

In conclusion, both ensemble models are highly effective, but the hyperparameter-tuned Gradient Boosting Classifier demonstrates a slightly stronger overall predictive capability for driver attrition.

Actionable Insights and Recommendations

Based on the feature importance analysis from both the Random Forest and Gradient Boosting models, we can derive the following key insights and recommendations for Ola to mitigate driver attrition:

Commonly Important Features (across both models):

1. **Total Business Value:** This consistently appears as a top predictor. Drivers contributing higher business value are either more likely to stay or their churn has a more significant impact. Ola should investigate the relationship further. Is low business value a precursor to churn? Or do high-value drivers leave for better opportunities?
 - **Recommendation:** Implement loyalty programs or performance incentives specifically for high-performing drivers (those with high 'Total Business Value') to

ensure their retention. For drivers with consistently low business value, consider targeted training or re-evaluation to improve performance and engagement before they decide to leave.

2. **Income:** Driver income also plays a crucial role. This is a direct financial incentive.
 - **Recommendation:** Review compensation structures and ensure competitive pay. Consider performance-based bonuses or regular income reviews to address financial satisfaction. The `Income_Increased` feature being less important than raw `Income` suggests that a stable, good income is more critical than just an increase at some point.
3. **Grade / Joining Designation / Education_Level:** These features are related to the driver's professional standing and qualifications. 'Grade' in particular seems important.
 - **Recommendation:** Develop clear career progression paths and opportunities for skill development. Promote drivers based on performance and provide avenues for higher 'Grade' or 'Joining Designation' to foster a sense of growth and value within the company.
4. **Age:** Age frequently appears as an influential factor.
 - **Recommendation:** Conduct an analysis of churn rates across different age groups. Tailor retention strategies to specific age demographics; for instance, younger drivers might value career development, while older drivers might prioritize stability or benefits.
5. **Quarterly Rating:** The average quarterly rating is also a significant feature.
 - **Recommendation:** Implement a robust feedback system related to quarterly ratings. Provide constructive feedback and support to drivers with lower ratings to help them improve. Recognize and reward drivers with consistently high ratings.

Less Important but Still Relevant Features:

- **City-specific indicators:** While `City` was one-hot encoded, individual city features did not consistently rank as highly as core performance or demographic features. This suggests that while location might have some influence, it's overshadowed by personal performance and compensation.

Overall Strategic Recommendations:

- **Proactive Retention Programs:** Identify drivers at high risk of attrition (based on predictive model scores) and engage them with personalized retention efforts, such as direct communication, surveys to understand grievances, or targeted incentives.
- **Holistic Driver Engagement:** Beyond financial incentives, focus on creating a positive work environment, providing development opportunities, and ensuring drivers feel valued and supported.
- **Continuous Monitoring:** Regularly monitor the model's performance and retrain it with fresh data to ensure its predictions remain accurate and relevant as market conditions and driver demographics evolve.

By focusing on these key areas, Ola can develop targeted strategies to improve driver satisfaction and significantly reduce attrition rates.